

Rapport de stage

Simulation d'un robot de type PUMA

Réalisé par :

Erwan MAUGERI
Charles-Antoine MAISONNEUVE

Dans le cadre des formations :

Licence Double Diplôme Mathématiques Physique et Applications
Licence de Physique

Période du stage :

1 juin - 26 juin 2026

Lieu :

LISV

Encadrants :

M. Barthélémy CAGNEAU - M. Lahouari BENABOU

Table des matières

Remerciements	2
Introduction	3
1 Anatomie générale	4
1.1 Vocabulaire	4
1.2 Fixer les géométries du problème	5
2 Validation des méthodes	6
2.1 Modèle simple à 2 dimensions	6
2.2 Workspace	7
3 Étude géométrique	9
3.1 Géométrie directe	9
3.2 Géométrie directe avec matrices Denavit-Hartenberg (DH)	11
3.3 Workspace	13
3.3.1 Par géométrie inverse	14
3.3.2 Par échantillonnage (expérimentation d'une méthode réelle)	14
3.3.3 Implémentation et résultats	15
4 Cinématique et suivi de trajectoires	17
4.1 En théorie	17
4.2 En pratique	18
4.3 Comparaison de correcteur K	19
4.3.1 Même condition initiale : capacité à rester sur la trajectoire	21
4.3.2 Conditions initiales différentes : capacité à rattraper une trajectoire	24
Conclusion	25
Bibliographie et sources	26
A Workspace modèle simplifié (en 2D)	27
B Modèle de géométrie directe (MGD)	29
C Fonction de géométrie inverse	31
D Acquisition des points du workspace (méthode par échantillonnage)	32
E Résultats d'acquisition du workspace	33
F Suivi de trajectoire (instantané)	35
G Suivi de trajectoire : preuve de la convergence en petit dt	36
H Suivi de trajectoire (code complet)	38
I Hodographes de la partie trajectoire carrée avec même condition initiale	44
J Hodographes de la partie cercle conditions initiales différentes	45
K Erreur partie cercle conditions initiales différentes pour K=100	46

Remerciements

Ce rapport présente la démarche suivie, les méthodes employées ainsi que les résultats obtenus au cours de ce stage. Sa réalisation n'aurait pas été possible sans le soutien d'un certain nombre de personnes que nous souhaitons remercier ici.

Nous tenons d'abord à adresser nos remerciements à Monsieur CAGNEAU, notre maître de stage, pour sa confiance et ses précieux conseils tout au long de cette expérience. Son encadrement a constitué un soutien essentiel dans la conduite de ce travail.

Nous remercions également l'ensemble des équipes du LISV pour leur accueil. Et nous adressons une mention particulière à Monsieur BENABOU (référent LISV) qui est resté disponible et à l'écoute tout au long du stage. Son aide, aussi bien sur les aspects administratifs que sur les difficultés techniques rencontrées, a grandement contribué au bon déroulement du stage.

(Erwan) Je souhaite aussi remercier Monsieur SCOLA, mon enseignant cette année dans les matières "Projet Numérique" et "Python pour la Physique". Les méthodes de travail rigoureuses et profondément actuelles qu'il nous a transmises ainsi que les projets réalisés dans le cadre de ses enseignements ont constitué une excellente préparation à ce stage. J'ai eu le sentiment de poursuivre et de mettre en application les compétences acquises dans ses cours, ce qui a rendu cette expérience particulièrement enrichissante.

Enfin, nous remercions l'ensemble des responsables de la licence pour leur investissement dans l'accompagnement des étudiants et pour les démarches mises en place afin de faciliter l'accès aux stages.

Introduction

La robotique occupe aujourd'hui une place essentielle dans de nombreux domaines industriels, notamment pour les tâches de manipulation, d'assemblage ou de déplacement précis. Les robots manipulateurs à plusieurs degrés de liberté permettent de réaliser des mouvements complexes tout en garantissant précision et répétabilité.

L'objectif de ce projet est de développer des simulations d'un modèle simplifié d'un robot manipulateur de type PUMA à l'aide du langage de programmation Python. Parmi ces simulations nous allons définir la géométrie du robot et son comportement, calculer sa position et l'orientation du robot dans l'espace à l'aide du modèle de géométrie directe et inverse, et réaliser un suivi de trajectoire à l'aide des modèles cinématiques.

Le PUMA (*Programmable Universal Manipulation Arm* ou "*bras programmable universel de manipulation*") a été développé dans les années 1970 pour répondre aux besoins croissants d'automatisation dans l'industrie manufacturière. Victor Scheinman est un roboticien qui a conçu le bras robotisé Stanford arm à l'université de Stanford. Il a mené un projet en collaboration avec l'entreprise Unimation, qui est considéré comme l'une des premières entreprises spécialisée dans les robots industriels, pour concevoir une nouvelle génération de robot articulés basés sur le bras humain afin de reproduire ces mouvements. Le premier modèle PUMA fut introduit vers 1978 et connut rapidement un grand succès car il est le premier robot à pouvoir effectuer une multitude de tâches différentes qui pouvait nécessiter de la précision ou de la force, son architecture est devenue le modèle des robots articulés modernes.

Le PUMA s'utilise aujourd'hui dans d'autres domaines que la manufacture, notamment en médecine, où une version modifiée du PUMA classique permet aux chirurgiens d'opérer à distance ; dans le cinéma et le sport télévisé où l'on a besoin d'effectuer des mouvements de caméras complexes et répétés avec du matériel vidéo souvent très lourd, et d'autres usages plus niche comme dans les parcs attractions ou les restaurants.

Toutes les unités seront exprimées dans le système international (en particulier en mètres [m] et en secondes [s]). On importera les modules python Numpy et Matplotlib de la manière suivante avant d'exécuter le code en annexe :

```
1 import numpy as np
2 import matplotlib.pyplot as plt
```

Toutes les figures sont originales sauf mention contraire en bibliographie.

1 Anatomie générale

1.1 Vocabulaire

Le PUMA désigne donc un standard de robots articulés. Il est capable de se déplacer dans les trois dimensions de l'espace et imite en substance l'anatomie humaine. On parlera donc de décalage de l'épaule, de poignet, de bras...

Commençons par distinguer les différentes pièces du PUMA (*voir en gras et fig 1.1*). Tout d'abord la **base/le socle** est la partie fixe du robot qui soutient la structure. Elle est reliée au bâti. Ensuite la rotation de la base, en liaison pivot avec le socle, c'est elle qui permet au bras de tourner de droite à gauche. L'ensemble est comparable à un buste.

Ensuite on entre dans le **bras**. L'arrière-bras (oui ça se dit ;) est le premier grand segment du bras. Il est relié à la rotation de la base par l'épaule une liaison pivot, la plupart du temps perpendiculaire à la précédente. Ensuite le coude, une liaison pivot à l'extrémité de l'arrière-bras, il relie l'arrière et l'avant-bras qui est la pièce suivante. Souvent l'arrière et l'avant-bras font la même taille, sinon l'avant-bras est plus court. L'avantage du "coude" sur un PUMA c'est qu'il est également déporté comme l'épaule. On peut donc réaliser des mouvements qu'un "coude naturel" ne permettrait pas comme toucher son arrière-bras (essayez de toucher votre biceps pour voir). C'est l'occasion de parler et d'introduire le concept de *limite articulaire*. Le coude humain est limité à :

- tendu : bras droit, arrière et avant-bras alignés
- plié : poing qui touche le deltoïde

Pour être beaucoup plus précis, l'angle de rotation du coude (qui constitue une liaison pivot) est empêché, il ne peut évoluer qu'entre 0° (bras tendu) ou $\approx 160^\circ$ (bras plié). Ce sont donc les limites articulaires du coude humain.

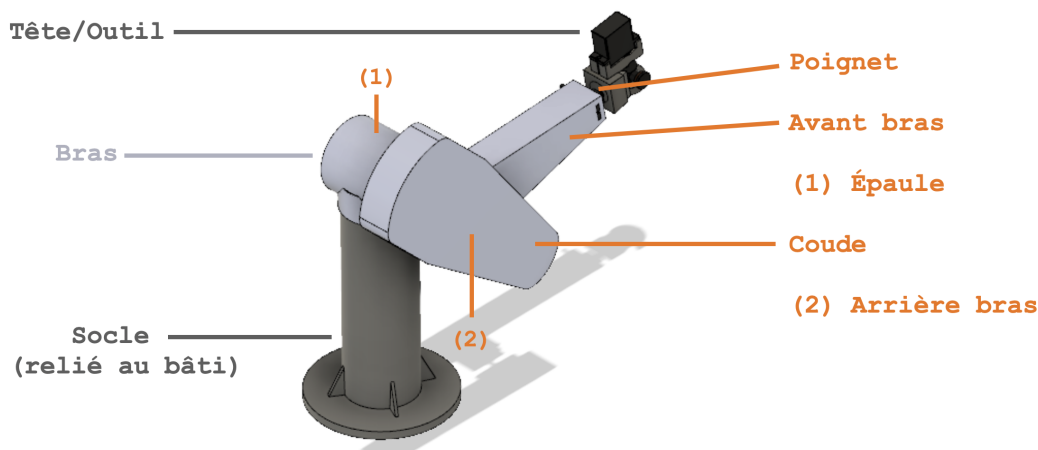


FIGURE 1.1 – Les différentes pièces du PUMA

L'avant-bras est la dernière pièce avant le système de **poignet**. C'est la deuxième partie du robot. On la désignera souvent par "outil" ou "tête". Sur le principe des angles d'Euler, le premier axe de rotation, dans le sens de l'avant-bras oriente l'outil. On parle de rotation du poignet. Sur un bras humain cette action se fait la rotation du radius autour de l'ulna (c'est donc une action qui naît dans l'avant-bras). Le deuxième axe de rotation

(inclinaison du poignet) contrôle le mouvement de bascule (droite/gauche ou haut/bas) du poignet. Le troisième complète les deux précédents.

On appelle effecteur terminal l'outil que l'on donne à manipuler au robot. Il varie fonction de l'utilisation. Nous n'en parlerons pas dans ce rapport.

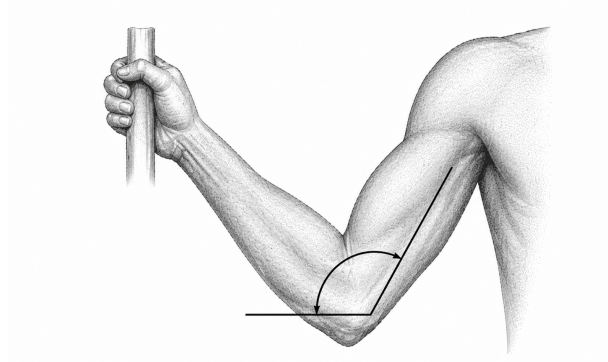


FIGURE 1.2 – Illustration de la limite articulaire du coude, le PUMA peut souvent alterner entre configuration bras gauche et bras droit, car son coude est rarement contraint comme le coude naturel (l'angle figuré peut être positif ou négatif).

Reste à définir comment s'orientent les liaisons pivots les unes par rapport aux autres, sont-elles parfaitement perpendiculaires ? légèrement gauches ? Au LISV nous avons pu tester une version sur laquelle les liaisons pivots, notamment au niveau du poignet sont légèrement désaxés. C'est utile notamment si l'effecteur est un objet long, ça déplace l'espace de travail (workspace) du poignet loin des bras du robot.

Pour notre simulation nous prendrons les axes des liaisons pivots perpendiculaires ou parallèles au précédent/suivant comme c'est le cas sur le schéma de la figure 1.3.

1.2 Fixer les géométries du problème

Il va maintenant falloir fixer les géométries du problème c'est-à-dire la longueur des différentes pièces, le nom des angles et dans un second temps les limites articulaires. On désigne la tête comme "l'outil" dans cette explication.

On note R_0 le repère absolu lié au socle puis R_1, R_2, R_3 les 3 repères intermédiaires liés aux différentes pièces du bras C_0, C_1, C_2, C_3 (C pour "corps", ce sont des solides rigides). On définit ensuite les vecteurs X_i, Y_i, Z_i de la base R_i et les q_i tel que $q_1 = (X_0, X_1)$, $q_2 = (X_1, X_2)$ et $q_3 = (X_2, X_3)$.

Notons d_1 la hauteur du socle, d_2 le décalage de l'épaule, a_2 la longueur de l'arrière-bras, d_3 le décalage du coude, a_4 la longueur de l'avant-bras.

La figure ci-après résume ce que l'on a dit. Pour une étude géométrique puis cinématique, nous pouvons considérer les différentes pièces comme étant des tiges sans épaisseur. L'outil est représenté par une pince. Gardons dans un coin de notre tête que les angles q_4, q_5 et q_6 existent.

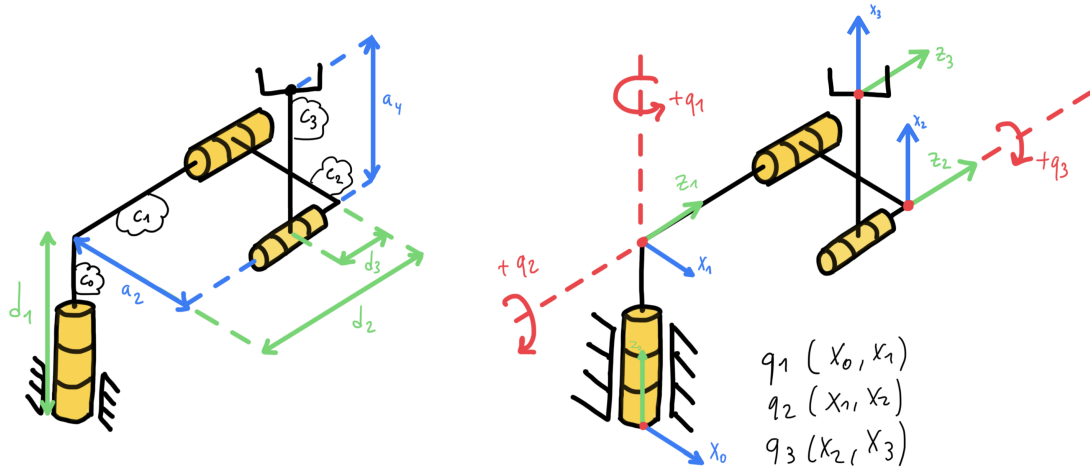


FIGURE 1.3 – Schématisation du PUMA

2 Validation des méthodes

2.1 Modèle simple à 2 dimensions

Commençons par étudier un modèle simple dans le plan. Considérons un manipulateur constitué de deux tiges rigides de longueurs $L_1 \geq L_2$, reliées par deux articulations pivot. Les positions de ces articulations sont décrites par les angles q_1 et q_2 . Ce modèle, bien que simplifié, permet d'introduire les principaux concepts de la robotique manipulatrice tels que la géométrie/cinématique directe/inverse et la notion d'espace de travail (workspace).

La géométrie s'intéresse à la description des positions et des orientations du robot dans l'espace, indépendamment du temps. À l'inverse, la cinématique prend en compte l'évolution temporelle de ces grandeurs et relie les vitesses articulaires aux vitesses du robot.

On distingue également les problèmes direct et inverse. La géométrie directe consiste à déterminer la position de l'effecteur à partir des variables articulaires. Mathématiquement, cette relation peut s'écrire :

$$x = F(q)$$

où q représente le vecteur des coordonnées articulaires et x la position de l'effecteur dans l'espace. Le problème inverse consiste quant à lui à déterminer les coordonnées articulaires permettant d'atteindre une position donnée. Il revient donc à résoudre :

$$q = F^{-1}(x)$$

lorsque cette inverse existe. Contrairement au problème direct, le problème inverse peut admettre plusieurs solutions, une unique solution ou aucune solution selon la position demandée et les contraintes du robot.

Chaque nouvelle articulation apporte un degré de liberté en plus au système. Un degré de liberté correspond à une variable indépendante nécessaire pour décrire complètement la configuration du robot. Pour un robot possédant n articulations, la configuration est donc définie par le vecteur articulaire :

$$q = (q_1, q_2, \dots, q_n)$$

Dans le cas du robot PUMA, plusieurs configurations articulaires différentes peuvent conduire à une même position de l'effecteur. Ainsi, pour une position atteignable donnée, il peut exister plusieurs solutions du problème de géométrie inverse, correspondant à différentes postures du bras (coude haut, coude bas, épaule droite, épaule gauche, etc.). Dans le cas de notre modèle simplifié à 2 bras il existe jusqu'à 2 solutions pour un point atteignable.

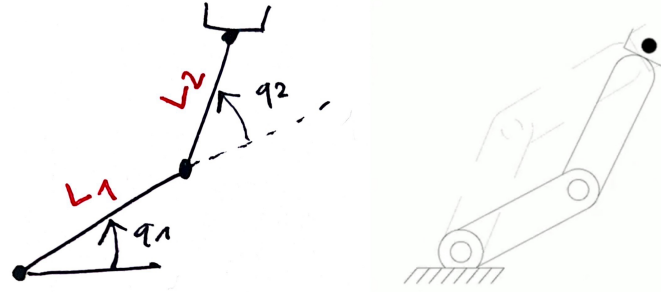


FIGURE 2.1 – Modèle à 2 tiges (gauche), il peut exister plusieurs solutions pour le même point (droite).

2.2 Workspace

En robotique on cherche donc la plupart du temps à atteindre un certain point en géométrie ou à appliquer une certaine vitesse en celui-ci si on est en cinématique. On travaille donc davantage avec le modèle inverse qu'avec le modèle direct.

En géométrie, on cherchera donc à savoir si le point cible est atteignable. Le problème sur le bras de type PUMA se résume comme suit : pour une position (x, y, z) existe-t-il une solution (q_1, q_2, q_3) tel que $F(q_1, q_2, q_3) = (x, y, z)$. Il peut y avoir plusieurs solutions selon les longueurs de bras et les contraintes articulaires comme il peut ne pas y avoir de solution, typiquement si le point est trop loin ou très proche du socle. L'ensemble des solutions à l'équation précédente constitue l'ensemble des positions atteignables et on appelle cela le *Workspace* ("espace de travail").

Dans notre modèle simple le workspace est "au mieux" un disque plein de rayon $L_1 + L_2$ si $L_1 = L_2$. Le workspace est donc plan (sans surprise). On imagine bien que le workspace du PUMA lui sera une boule plus ou moins pleine. Des "trous" apparaissent dans le workspace selon la longueur des bras mais surtout en fonction des limites articulaires.

Le modèle de géométrie directe est assez simple pour le bras à deux dimensions :

$$\begin{cases} x = L_1 \cos(q_1) + L_2 \cos(q_1 + q_2) \\ y = L_1 \sin(q_1) + L_2 \sin(q_1 + q_2) \end{cases} \quad (2.1)$$

Donc en isolant q_1 et q_2 on obtient en fonction de x et y :

$$\begin{cases} q_1 = \tan^{-1}\left(\frac{y}{x}\right) - \cos^{-1}\left(\frac{L_1^2 - L_2^2 + x^2 + y^2}{2L_1\sqrt{x^2 + y^2}}\right) \\ q_2 = \pi - \cos^{-1}\left(\frac{L_1^2 + L_2^2 - x^2 - y^2}{2L_1L_2}\right) \end{cases} \quad (2.2)$$

On implémente les deux fonctions q_1 et q_2 de géométrie inverse en Python mais également une fonction qui teste si le point entré par l'utilisateur est atteignable par le bras robot (annexe A).

Pour cela, on définit d'abord les paramètres géométriques du bras, à savoir les longueurs L_1 et L_2 , ainsi que les limites articulaires I_{q_1} et I_{q_2} , qui correspondent aux plages de valeurs autorisées pour les angles q_1 et q_2 . L'algorithme fonctionne alors de la manière suivante :

- Pour un point cible $M(x,y)$, on calcule sa distance à l'origine du repère, qui coïncide avec le point d'attache du bras robotique : $r = \sqrt{x^2 + y^2}$.
- On vérifie ensuite si ce point appartient au workspace théorique du bras, c'est-à-dire à l'ensemble des points atteignables en ne tenant compte que des longueurs des deux segments. Cette condition s'écrit : $|L_1 - L_2| \leq r \leq L_1 + L_2$
- Si cette condition n'est pas vérifiée, le point est situé en dehors de l'espace de travail théorique du bras et aucune solution n'existe.
- Dans le cas contraire, le point est géométriquement atteignable par les longueurs du bras. On calcule alors les angles q_1 et q_2 à l'aide des équations de géométrie inverse. Cependant, cette première solution doit encore être confrontée aux limites articulaires imposées au robot. On vérifie donc que $q_1 \in I_{q_1}$ et $q_2 \in I_{q_2}$

Si ces deux conditions sont satisfaites, alors le point demandé est effectivement atteignable par le bras robotique avec les contraintes considérées, et l'algorithme renvoie les valeurs des angles q_1 et q_2 . Dans le cas contraire, le point appartient au workspace théorique, mais il n'est pas atteignable compte tenu des limites articulaires choisies.

Voici enfin quelques cas intéressants, commentés, correspondant à des configurations classiques ou plus particulières :

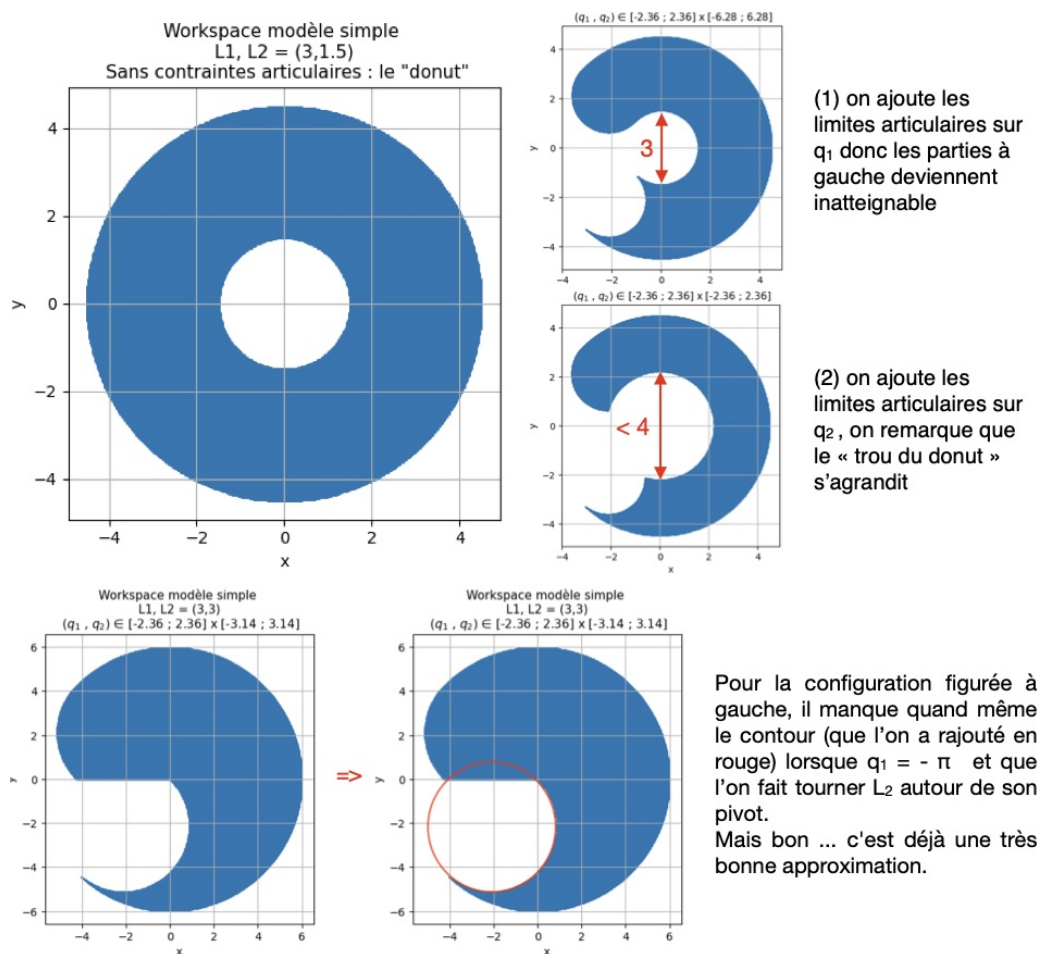


FIGURE 2.2 – Quelques Workspace en 2d

Ces résultats confirment les attentes, dans la mesure où le comportement du robot articulé a fait l'objet d'une modélisation bidimensionnelle (à deux segments) sur GeoGebra.

3 Étude géométrique

On peut maintenant revenir au problème initial du PUMA comme on l'a décrit pour la dernière fois figure 1.3.

3.1 Géométrie directe

Le modèle géométrique direct (MGD) permet de déterminer la position de l'effecteur du robot dans l'espace à partir des valeurs connues des articulations (les q_i). Pour établir cette relation, les transformations associées à chaque articulation sont composées successivement le long de la chaîne cinématique, depuis la base jusqu'à l'effecteur. Soient O, A, B, C, C', D les points suivants :

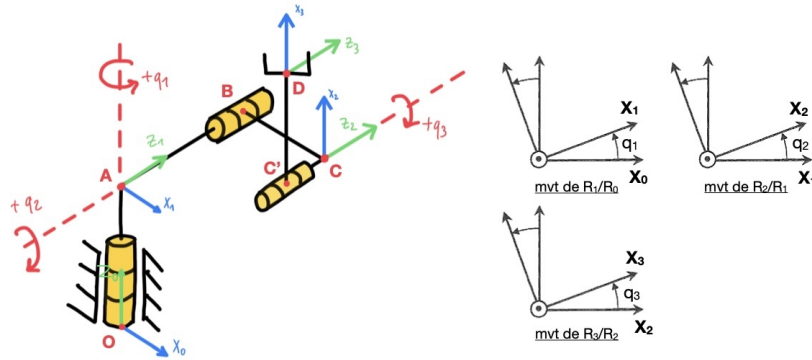


FIGURE 3.1 – Repères intermédiaires

Voici les calculs pour aboutir à $\overrightarrow{OE}$ en fonction des q_i .

$$\overrightarrow{OA} = \begin{pmatrix} 0 \\ 0 \\ d_1 \end{pmatrix}_{R_0} \quad (3.1)$$

$$\overrightarrow{OB} = \begin{pmatrix} d_2 \cos(q_1) \\ d_2 \sin(q_1) \\ d_1 \end{pmatrix}_{R_0} \quad (3.2)$$

$$\begin{aligned} \overrightarrow{BC} &= \begin{pmatrix} a_2 \cos(q_2) \\ a_2 \sin(q_2) \\ 0 \end{pmatrix}_{R_1} = a_2 \cos(q_2) [\cos(q_1) \vec{X}_0 + \sin(q_1) \vec{Y}_0] - a_2 \sin(q_2) \vec{Z}_0 \\ &= \begin{pmatrix} a_2 \cos(q_1) \cos(q_2) \\ a_2 \sin(q_1) \cos(q_2) \\ -a_2 \sin(q_2) \end{pmatrix}_{R_0} \end{aligned}$$

Donc :

$$\overrightarrow{OC} = \begin{pmatrix} d_2 \cos(q_1) + a_2 \cos(q_1) \cos(q_2) \\ d_2 \sin(q_1) + a_2 \sin(q_1) \cos(q_2) \\ d_1 - a_2 \sin(q_2) \end{pmatrix}_{R_0} \quad (3.3)$$

1^{ère} approximation : $\overrightarrow{CC'} = \overrightarrow{0}$. Donc $\overrightarrow{OC} = \overrightarrow{OC'}$ ou plus simplement $C = C'$.

$$\overrightarrow{CD} = \begin{pmatrix} a_4 \cos(q_3) \\ a_4 \sin(q_3) \\ 0 \end{pmatrix}_{R_2}$$

Or : $\overrightarrow{X}_2 = \cos(q_2)\overrightarrow{X}_1 + \sin(q_2)\overrightarrow{Y}_1$, $\overrightarrow{Y}_2 = -\sin(q_2)\overrightarrow{X}_1 + \cos(q_2)\overrightarrow{Y}_1$ et $\overrightarrow{Z}_2 = \overrightarrow{Z}_1$

Donc :

$$\overrightarrow{CD} = a_4 \cos(q_3) [\cos(q_2)\overrightarrow{X}_1 + \sin(q_2)\overrightarrow{Y}_1] + a_4 \sin(q_3) [-\sin(q_2)\overrightarrow{X}_1 + \cos(q_2)\overrightarrow{Y}_1]$$

$$= \begin{pmatrix} a_4 [\cos(q_3)\cos(q_2) - \sin(q_3)\sin(q_2)] \\ a_4 [\cos(q_3)\sin(q_2) + \sin(q_3)\cos(q_2)] \\ 0 \end{pmatrix}_{R_1} = \begin{pmatrix} a_4 \cos(q_2 + q_3) \\ a_4 \sin(q_2 + q_3) \\ 0 \end{pmatrix}_{R_1}$$

$$= a_4 \cos(q_2 + q_3) [\cos(q_1)\overrightarrow{X}_0 + \sin(q_1)\overrightarrow{Y}_0] - a_4 \sin(q_2 + q_3) \overrightarrow{Z}_0$$

$$\overrightarrow{CD} = \begin{pmatrix} a_4 \cos(q_2 + q_3) \cos(q_1) \\ a_4 \cos(q_2 + q_3) \sin(q_1) \\ -a_4 \sin(q_2 + q_3) \end{pmatrix}_{R_0}$$

$$\overrightarrow{OD} = \overrightarrow{OC} + \overrightarrow{CD} = \begin{pmatrix} d_2 \cos(q_1) + a_2 \cos(q_1) \cos(q_2) + a_4 \cos(q_2 + q_3) \cos(q_1) \\ d_2 \sin(q_1) + a_2 \sin(q_1) \cos(q_2) + a_4 \cos(q_2 + q_3) \sin(q_1) \\ d_1 - a_2 \sin(q_2) - a_4 \sin(q_2 + q_3) \end{pmatrix}_{R_0} \quad (3.4)$$

En python, on écrit les fonctions qui correspondent au modèle de géométrie directe (MGD) décrit par les équations 3.1 à 5 (voir annexe B). On fait en sorte que les fonctions fonctionnent aussi bien pour des nombres isolés - on envoie (q_1, q_2, q_3) et on reçoit (x, y, z) - que pour des tableaux entiers - on envoie une série de (q_1, q_2, q_3) : $[[q_{1,1}, q_{1,2}, q_{1,3}], [q_{2,1}, q_{2,2}, q_{2,3}], \text{etc}]$ et on reçoit une série de (x, y, z) : $[[x, y, z], [x, y, z], \text{etc}]$. On peut réaliser des animations de déplacements avec des boucles et en enregistrant chaque image ("frame") et grâce à la commande `plt.savefig()` de Python.

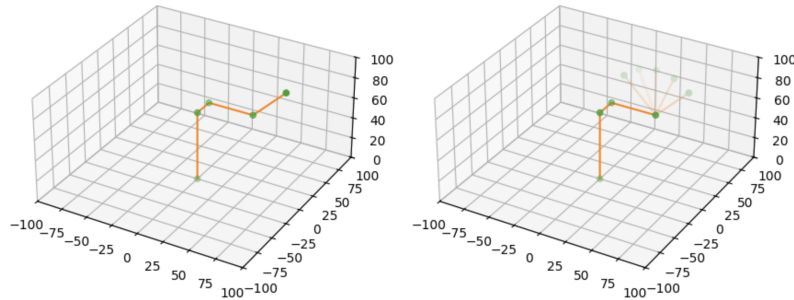


FIGURE 3.2 – Schémas avec MGD (de gauche à droite) : position de repos $q_i = 0$, "Salut" de la main en faisant osciller q_3 entre -0,8 et 0,8 (empilement).

3.2 Géométrie directe avec matrices Denavit-Hartenberg (DH)

Constat

Jusqu'à maintenant pour la géométrie directe nous avons dessiné les repères R_0, R_1, R_2, R_3 de manière intuitive, puis projeté manuellement les vecteurs les uns sur les autres (par exemple, en exprimant $\overrightarrow{CD}$ du repère R_2 puis dans R_1 , puis dans R_0). L'avantage c'est que l'on comprend parfaitement la physique et la géométrie du bras en le faisant. L'inconvénient c'est qu'il est très facile de faire une erreur de signe ($\pm \sin$ ou $\pm \cos$) lors des projections à la main sur papier.

Ce que fait Denavit-Hartenberg (La méthode algorithmique)

La méthode DH impose des règles ultra-strictes pour placer les repères. Par exemple : l'axe $\overrightarrow{X_i}$ doit obligatoirement être perpendiculaire à $\overrightarrow{Z_{i-1}}$ et $\overrightarrow{Z_i}$. Une fois ces repères placés selon la règle, on n'a plus besoin de réfléchir ou de projeter des forces/vecteurs sur un dessin. On a juste à :

- Remplir un tableau avec les 4 paramètres $(d_i, \theta_i, a_i, \alpha_i)$ pour chaque segment.
- Remplacer ces valeurs dans la matrice générique.
- Laisser l'ordinateur multiplier les matrices entre elles.

Cette méthode sera aussi très utile pour passer à l'analyse cinématique directe puis à la géométrie et cinématique inverse. On reconnaît que cette méthode donne des paramétrisations moins naturelles mais le gain dans les calculs est significatif.

La méthode

Pour définir une transformation entre deux repères, il faut 6 paramètres (3 rotations, 3 translations). La méthode de Denavit-Hartenberg permet de définir la transformation grâce à quatre paramètres. Pour cela il faut définir les axes selon certaines règles :

- l'axe Z_i est porté par l'axe de la liaison reliant le corps C_{i-1} au corps C_i
- l'axe X_i est porté par la normale commune à Z_{i-1} et Z_i donc : $X_i = Z_{i-1} \wedge Z_i$
- l'axe Y_i est choisi à ce que le repère soit direct : $Y_i = Z_i \wedge X_i$

La méthode Denavit-Hartenberg décompose le passage d'un repère $(i-1)$ au suivant (i) en 4 mouvements élémentaires : 2 translations et 2 rotations. La particularité de la méthode de Denavit-Hartenberg (DH) est donc qu'elle exclut l'axe Y du calcul.

On positionne alors les repères de manière à ce que tous les déplacements se fassent uniquement selon les axes X et Z (l'axe Y servant uniquement à fermer le repère [règle de la main droite] on ne le fait pas figurer sur le schéma). Chaque liaison est ainsi définie par 4 paramètres :

- d_i : Translation selon Z_{i-1}
- θ_i : Rotation autour de Z_{i-1} (angle entre X_{i-1} et X_i)
- a_i : Translation selon X_i
- α_i : Rotation autour de X_i (angle entre Z_{i-1} et Z_i)

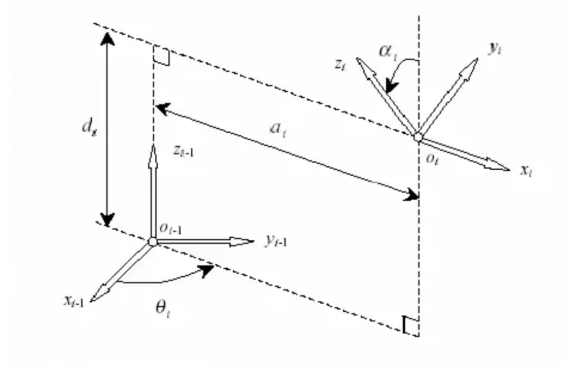


FIGURE 3.3 – Relation entre le repère i et le repère $i - 1$

On construit alors le tableau suivant (les 3 premières lignes uniquement) :

	i	d_i (décalage selon z)	θ_i (rotat° autour de z)	a_i (... selon x)	α_i (... autour de x)
${}^0T^1$	1	d_1	q_1	0	-90°
${}^1T^2$	2	d_2	q_2	a_2	0°
${}^2T^3$	3	$-d_3$	q_3	a_4	0°
${}^3T^4$	4	0	0	0	q_4
${}^4T^5$	5	0	q_5	a_5	0
${}^5T^6$	6	0	0	0	q_6

TABLE 3.1 – Paramètres de Denavit-Hartenberg du modèle PUMA simplifié

Et on écrit les matrices ${}^{i-1}T^i$ en suivant le modèle ci-dessous (dont on ne fera pas la démonstration ici) :

$${}^{i-1}T^i = \begin{bmatrix} \cos \theta_i & -\cos \alpha_i \sin \theta_i & \sin \alpha_i \sin \theta_i & a_i \cos \theta_i \\ \sin \theta_i & \cos \alpha_i \cos \theta_i & -\sin \alpha_i \cos \theta_i & a_i \sin \theta_i \\ 0 & \sin \alpha_i & \cos \alpha_i & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Remarque : Ces matrices fonctionnent en chaîne. Admettons que l'on veuille connaître la position du bras 3 :

$${}^0T^3(q_1, q_2, q_3) = {}^0T^1(q_1) \times {}^1T^2(q_2) \times {}^2T^3(q_3)$$

Cas de la tête et application

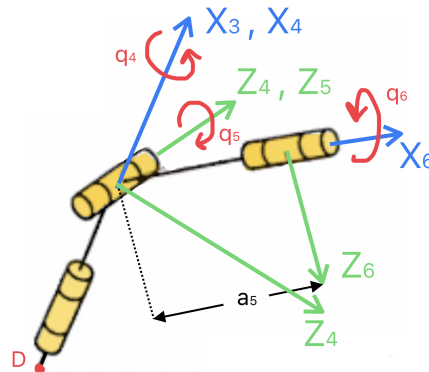


FIGURE 3.4 – Schématisation de "l'outil"/"la tête" du bras PUMA

Continuons d'écrire nos matrices DH en s'attaquant à la partie "outil"/tête du bras PUMA. Le paramétrage choisi pour la tête est détaillé en figure 3.4.

On rajoute les 3 dernières lignes du tableau 3.1 (c'est-à-dire les lignes 4,5,6) et on peut écrire le programme suivant pour calculer chaque matrice de DH :

```

1 def T_i(di, theta_i, ai, alpha_i):
2     # represente T de i-1 vers i
3     return np.array([
4         [np.cos(theta_i), -np.cos(alpha_i)*np.sin(theta_i), np.sin
5         (alpha_i)*np.sin(theta_i), ai*np.cos(theta_i)],
6         [np.sin(theta_i), np.cos(alpha_i)*np.cos(theta_i), -np.sin
7         (alpha_i)*np.cos(theta_i), ai*np.sin(theta_i)],
8         [0, np.sin(alpha_i), np.cos(alpha_i), di],
9         [0, 0, 0, 1]
10    ])

```

```

1 # exemples
2 T_1 = T_i(d1, q1, 0, -np.pi/2)
3 T_2 = T_i(d2, q2, a2, 0)

```

3.3 Workspace

Le PUMA est donc un bras robotique composé exclusivement de pivots (il n'y a pas de glissière dans un robot PUMA standard) où chaque pivot correspond à un degré de liberté. Pour chaque position atteignable il existe nombre parfois infini de solutions (surtout si on impose pas d'orientation de l'outil comme ce sera le cas dans cette partie). Voici ce qu'il faut retenir :

- Comme avec le modèle simplifié en 2d, il faudra distinguer le cas bras droit / bras gauche.
- Si le nombre de degrés de liberté (DDL) est strictement supérieur au nombre de contraintes imposées, il y a généralement une infinité de solutions (redondance). Par exemple, avec un bras à 3 DDL, imposer seulement la position (x, y) revient à imposer 2 contraintes pour 3 inconnues, ce qui conduit généralement à une infinité de solutions.
- Si le nombre de DDL est égal au nombre de contraintes imposées, il y a généralement un nombre fini de solutions (0, 1, 2, ... selon la géométrie et la position demandée). Par exemple, avec un bras à 3 DDL, imposer la position et l'orientation de l'effecteur (x, y, ϕ) revient à imposer 3 contraintes pour 3 inconnues, ce qui conduit généralement à un nombre fini de solutions.

À partir de maintenant et pour le reste du compte rendu nous prendrons les tailles de pièce et limites articulaires suivantes pour le bras PUMA :

```

1 # Valeurs types PUMA 560 (en m)
2 d1 = 0.660
3 d2 = 0.149
4 a2 = 0.432
5 d3 = 0
6 a4 = 0.433
7
8

```

```

9 # Limites articulaires
10 I_q1 = (-2.79, 2.79)
11 I_q2 = (-3.93, 0.79)
12 I_q3 = (-3.93, 0.79)

```

Le but de cette partie sera de construire une fonction vérifiant que le "point cible" est atteignable par le bras, mais également de représenter le workspace sous forme d'un nuage de points en 3 dimensions. On s'attend à trouver une boule partiellement pleine (selon les limites articulaires) et légèrement désaxée (à cause du décalage de l'épaule).

3.3.1 Par géométrie inverse

Une première approche pour déterminer le workspace est par géométrie inverse. On fait comme avec le modèle simplifié 2d et on exprime (à l'aide des équations de géométries inverse 3.1à4) les q_i en fonction de x, y, z :

$$q_1 = \text{atan2}(y, x) - \text{atan2}\left(d_2, \pm\sqrt{x^2 + y^2 - d_2^2}\right) \quad (3.5)$$

$$q_2 = \text{atan2}(AZ - BR, AR + BZ) \quad (3.6)$$

$$q_3 = \text{atan2}\left(D, \pm\sqrt{1 - D^2}\right) \quad (3.7)$$

Variables intermédiaires :

$$R = x \cos(q_1) + y \sin(q_1)$$

$$Z = z - d_1$$

$$D = \frac{a_2^2 + a_4^2 - (R^2 + Z^2)}{2a_2a_4}$$

$$A = a_2 - a_4 \sin(q_3)$$

$$B = a_4 \cos(q_3)$$

On écrit ensuite la fonction "q" (voir annexe C qui calcule le modèle géométrique inverse si et seulement si il existe une solution pour la position demandée. Elle prend donc en paramètres q_1, q_2, q_3 et renvoie x, y, z si et seulement si cette position est atteignable. C'est donc une fonction 2 en 1 qui vérifie que la position demandée est atteignable conformément à la géométrie du bras et aux contraintes articulaires imposées. Voici les cas d'utilisation de la-dite fonction :

```

1 # un test qui marche (la position (x,y,z) est atteignable)
2 print(q(0.05, 0.015, 0.066, s1=1, s3=1))
3 # un test qui ne marche pas (hors workspace)
4 print(q(0, 0, 0, s1=1, s3=1))

```

Sorties :

(0.00199, -0.955, 0.337)

Erreur : x, y hors de portée

3.3.2 Par échantillonnage (expérimentation d'une méthode réelle)

En théorie le modèle de géométrie inverse nous suffit mais il demande beaucoup de calculs et peut donc parfois être assez lent (surtout sur des ordinateurs peu puissants de systèmes embarqués). Une méthode numérique plus simple et suffisante dans la grande majorité des cas est de comparer la position cible/désirée (entrée par l'utilisateur) avec des positions connues, dont on sait par avance qu'elles sont atteignables. On va donc

échantillonner nos q_i pour générer cette "base de données". L'avantage est que l'on va pouvoir dans le même temps tracer tous ces points dans un graphe 3d et obtenir le fameux "nuage de points" qui permet de représenter visuellement le workspace.

Une fois le nuage de points du workspace généré, il est possible d'utiliser celui-ci pour estimer rapidement l'atteignabilité d'une position cible ($P_c = (x_c, y_c, z_c)$).

Soit $(P_1, P_2, \dots, P_N)$ l'ensemble des positions obtenues par échantillonnage des variables articulaires admissibles. Pour une cible donnée, on recherche le point du workspace le plus proche au sens de la distance euclidienne :

$$d_{\min} = \min_{i \in \{1, \dots, N\}} ||P_c - P_i||$$

Cette quantité représente la distance entre la position désirée et la position atteignable la plus proche connue dans notre base de données. Le critère d'atteignabilité retenu est alors :

$$d_{\min} < \varepsilon$$

où (ε) représente la tolérance maximale admissible. Si cette condition est vérifiée, la cible est considérée comme atteignable. Dans le cas contraire, elle est considérée comme extérieure au workspace échantillonné.

Le choix de (ε) dépend directement de la finesse du maillage articulaire utilisé. Si les articulations sont discrétisées avec un pas angulaire maximal (Δq) (en radians) et si la longueur totale du bras vaut

$$L_{\text{tot}} = \sum_i L_i,$$

alors l'erreur spatiale induite par la discrétisation peut être estimée par :

$$\varepsilon \approx L_{\text{tot}} \Delta q.$$

Cette relation fournit un ordre de grandeur cohérent : plus le maillage articulaire est fin, plus la valeur de (ε) peut être réduite. À l'inverse, un maillage grossier nécessite une tolérance plus importante afin de compenser les écarts entre les points du workspace échantillonné.

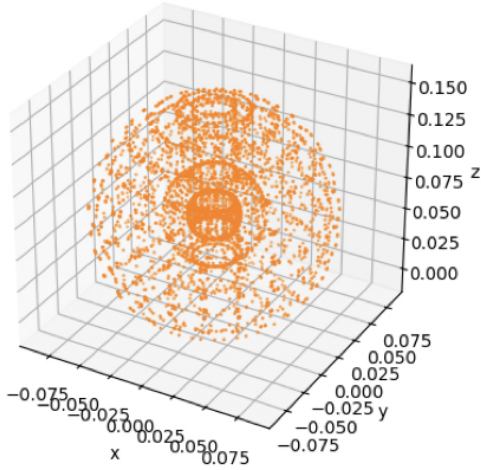
Cette méthode ne constitue pas une preuve mathématique d'atteignabilité mais une approximation numérique dont la précision dépend directement de la densité d'échantillonnage choisie. Donc elle est valable.

3.3.3 Implémentation et résultats

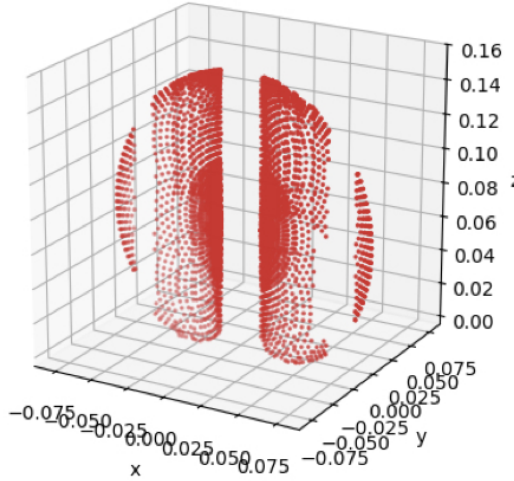
Consulter l'annexe D pour le programme d'acquisition. Il est conceptuellement assez simple : une triple boucle "for" pour q_1 , q_2 et q_3 sur le principe du dénombrement.

La figure 3.5 présente une série de nuages de points. Le premier (en haut à gauche, en orange) a un but purement esthétique : des valeurs faibles de N_{q_1} , N_{q_2} et N_{q_3} ont été choisies pour obtenir un nuage de faible densité permettant de voir à travers. On peut apprécier le fait que certaines zones sont plus denses que d'autres (remarquez notamment le cylindre vide). En contraignant au maximum les articulations, on obtient le nuage vert en haut à droite, où certaines zones deviennent totalement inaccessibles. Les deux derniers nuages sont des plans de coupe de configurations intéressantes (explicitées directement sur la figure). Le plan rouge intègre les limites articulaires définies. Le plan violet restreint davantage la plage des q_i , transformant le cylindre central vide en une sphère. Dans les deux cas, l'espace de travail est irrégulier et dépourvu de symétrie de révolution.

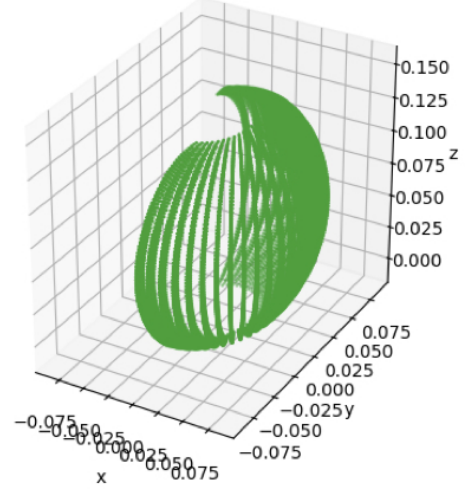
Workspace obtenu par échantillonnage
 $[N(q_1) = 20 ; N(q_2) = 20 ; N(q_3) = 10]$
 $d_1 = 0.066 ; d_2 = 0.0149 ; a_2 = 0.0432 ; d_3 = 0 ; a_4 = 0.0433$
 $l_{q1} = (-2.79, 2.79) ; l_{q2} = (-3.93, 0.79) ; l_{q3} = (-3.93, 0.79)$



Coupe verticale du workspace
 $[N_{q1} = 50 ; N_{q2} = 50 ; N_{q3} = 50]$



Workspace obtenu par échantillonnage
 $[N(q_1) = 30 ; N(q_2) = 30 ; N(q_3) = 30]$
 $d_1 = 0.066 ; d_2 = 0.0149 ; a_2 = 0.0432 ; d_3 = 0 ; a_4 = 0.0433$
 $l_{q1} = (-1.79, 1.79) ; l_{q2} = (-0.79, 0.79) ; l_{q3} = (-0.79, 0.79)$



Coupe verticale du workspace
 $[N_{q1} = 50 ; N_{q2} = 50 ; N_{q3} = 50]$
 $(q_1, q_2, q_3) \in [-2, 2] \times [-3.93, 0.79] \times [-1.79, 1.79]$

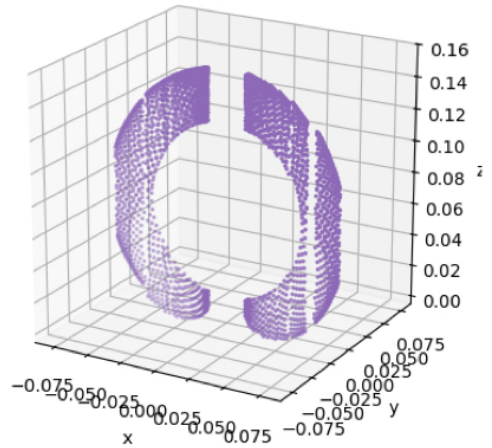


FIGURE 3.5 – Représentations de workspace d'un robot de type PUMA

Pour choisir les N_{qi} (pour l'échantillonnage) intéressants, on cherche à minimiser ε .
Or $\varepsilon \approx L_{tot} \Delta q$ avec

$$\Delta q = \max(\Delta q_1, \Delta q_2, \Delta q_3)$$

et

$$\Delta q_1 = \frac{q_{1,max} - q_{1,min}}{N_{q1} - 1}$$

$$\Delta q_2 = \frac{q_{2,max} - q_{2,min}}{N_{q2} - 1}$$

$$\Delta q_3 = \frac{q_{3,max} - q_{3,min}}{N_{q3} - 1}$$

et

$$L_{tot} = d_1 + d_2 + a_2 + d_3 + a_4 = 0.1674 [m]$$

On écrit ce petit programme pour trouver ε :

```

1 eps = 10
2 N = 2
3 while eps > 0.03 : # on veut epsilon < 3 cm
4     delta_q = max(
5         np.abs(I_q1[-1] - I_q1[0]) / (N - 1),
6         np.abs(I_q2[-1] - I_q2[0]) / (N - 1),
7         np.abs(I_q3[-1] - I_q3[0]) / (N - 1)
8     )
9     eps = delta_q * 0.1674
10    N+=1
11 print(eps, N)

```

Le programme retourne : 0.029190 34

L'acquisition de notre workspace d'étude avec $N_{q1} = N_{q2} = N_{q3} = 34$ est disponible en annexe E. On y joint un programme qui (sur le modèle du programme de géométrie inverse) nous dit si le point P (défini par l'utilisateur) est dans le workspace ou pas à l'aide du critère de tolérance décrit précédemment.

Maintenant que le workspace est connu, il nous est possible de définir des trajectoires réalisables. Pour cela, les trajectoires sont échantillonnées à l'intérieur du workspace, ce qui nous assure que chacun des points de passage est atteignable par le robot.

4 Cinématique et suivi de trajectoires

4.1 En théorie

Cette étape consiste au déplacement en temps réel de l'effecteur vers une cible connue. Contrairement à l'étude géométrique, qui s'intéresse uniquement aux positions du robot, la cinématique prend en compte l'évolution du mouvement au cours du temps. L'objectif n'est donc plus seulement de déterminer où se trouve l'effecteur, mais également de calculer comment le robot doit se déplacer pour atteindre une position donnée.

Pour la cinématique de notre robot, la vitesse est déterminée grâce à l'écart de position entre la cible et l'effecteur, noté e , ainsi qu'avec une constante K dont l'importance sera démontrée ultérieurement, par la formule suivante :

$$v = K \cdot e$$

où :

- e représente le vecteur d'erreur entre la position cible et la position actuelle de l'effecteur
- K est un coefficient de gain
- v représente la vitesse cartésienne souhaitée de l'effecteur

Ainsi, plus le robot est éloigné de sa cible, plus la vitesse calculée sera importante. À l'inverse, lorsque l'effecteur se rapproche de la position désirée, la vitesse diminue progressivement jusqu'à tendre vers zéro.

Les moteurs du robot ne sont cependant pas commandés directement par cette vitesse cartésienne. Les variables de commande sont les vitesses articulaires des différents pivots

du robot :

$$\dot{q} = \begin{pmatrix} \dot{q}_1 \\ \dot{q}_2 \\ \dot{q}_3 \end{pmatrix}$$

Il est donc nécessaire de convertir la vitesse de l'effecteur en vitesses articulaires. Cette relation est donnée par la matrice jacobienne :

$$v = J(q)\dot{q}$$

où $J(q)$ dépend de la configuration instantanée du robot.

La matrice jacobienne est obtenue en dérivant les équations du modèle géométrique direct par rapport aux variables articulaires. Elle établit ainsi un lien direct entre les vitesses des articulations et la vitesse de déplacement de l'effecteur dans l'espace.

En inversant cette relation, il est possible de calculer les vitesses articulaires nécessaires pour obtenir une vitesse donnée de l'effecteur :

$$\dot{q} = J^{-1}(q)v$$

La jacobienne permet également d'identifier les singularités du robot. Une singularité correspond à une configuration particulière dans laquelle le robot perd une direction de mouvement indépendante. Mathématiquement, cela se traduit par :

$$\det(J) = 0$$

Dans ce cas, la matrice jacobienne n'est plus inversible et certaines vitesses cartésiennes ne peuvent plus être produites par le robot.

Afin d'éviter les problèmes numériques liés à ces situations, nous utilisons dans notre programme la pseudo-inverse de Moore-Penrose de la jacobienne. Les vitesses articulaires sont alors calculées à partir de :

$$\dot{q} = J^+v$$

où J^+ désigne la pseudo-inverse de la matrice jacobienne.

Cette méthode présente l'avantage de fournir une solution même lorsque la matrice jacobienne est proche d'une singularité. Elle est largement utilisée en robotique pour les algorithmes de suivi de trajectoire en temps réel.

4.2 En pratique

Dans notre étude, seuls les trois premiers axes du robot ont été conservés. En effet, la structure du robot PUMA permet de découpler le problème de la position et de l'orientation : les trois derniers axes du robot PUMA interviennent dans l'orientation de l'effecteur de l'effecteur alors que l'objectif de notre travail est uniquement de contrôler la position d'un point dans l'espace.

Le suivi de trajectoire repose sur une succession d'étapes répétées à chaque itération :

1. Calcul de la position actuelle de l'effecteur à l'aide du modèle géométrique direct
2. Calcul de l'erreur entre la position cible et la position actuelle

3. Détermination de la vitesse souhaitée
4. Calcul de la matrice jacobienne
5. Calcul des vitesses articulaires à l'aide de la pseudo-inverse de la jacobienne
6. Mise à jour des angles articulaires
7. Répétition du processus jusqu'à la fin de la trajectoire.

On met en annexe [F](#) un programme sans boucle qui décrit ces étapes et un schéma bloc.

À chaque instant, le modèle géométrique direct fournit la position actuelle du robot :

$$P = \begin{pmatrix} x \\ y \\ z \end{pmatrix}$$

Cette position est comparée à la position cible :

$$C = \begin{pmatrix} x_c \\ y_c \\ z_c \end{pmatrix}$$

L'erreur de suivi est alors définie par le vecteur :

$$e = C - P$$

Cette erreur permet de calculer la vitesse désirée de l'effecteur selon la loi de commande :

$$v = K \cdot e$$

où K représente le correcteur proportionnel, ou "gain".

Les vitesses articulaires sont ensuite calculées à partir de la pseudo-inverse de la matrice jacobienne :

$$\dot{q} = J^+ v$$

Les angles articulaires sont alors mis à jour à l'aide d'un pas de temps dt :

$$q_i(t + dt) = q_i(t) + \dot{q}_i dt$$

avec $i = 1, 2, 3$.

Au cours de la simulation, les positions successives de l'effecteur ainsi que celles de la cible sont enregistrées. Ces données permettent ensuite de tracer les trajectoires obtenues et d'évaluer visuellement la qualité du suivi.

4.3 Comparaison de correcteur K

Le correcteur K joue un rôle fondamental dans le comportement du système puisqu'il détermine directement la vitesse de correction de l'erreur.

Lorsque K est faible, la vitesse calculée reste modérée. Le mouvement du robot est alors très stable mais le suivi de trajectoire devient moins précis, notamment lorsque la cible se déplace rapidement. L'effecteur présente alors un retard important par rapport à la trajectoire théorique.

À l'inverse, lorsque K est élevé, les corrections sont plus rapides et le robot suit davantage la trajectoire imposée. Cependant, les vitesses articulaires deviennent plus importantes et peuvent générer des mouvements brusques ou des oscillations.

C'est le moment de parler de ce que l'on appellera "l'erreur signée". Jusqu'à maintenant on a parlé de l'erreur c'est-à-dire du décalage entre le point P et le point C instantané. Le truc c'est que parfois ; selon la vitesse, le gain de commande (on en reparlera), le point P dépasse le point C. Alors bien sûr tout cela est bien relatif mais se comprend plus ou moins intuitivement. Il faut donc penser à regarder l'angle entre le vecteur tangent à la trajectoire en C et le vecteur vitesse de commande. Si l'angle est aigu c'est bon signe, ça veut dire que le point P est bien en train de suivre le point C (on peut même dire qu'il est "en retard"). Si l'angle est obtus par contre, ça veut dire que le point P a dépassé le point C. Il va donc devoir faire demi-tour ou ralentir ce qui n'est pas très optimal. Cela peut arriver si C "freine", change brusquement de trajectoire ou alors si P est trop rapide. Ce concept d'erreur signée va donc nous être très utile pour comparer différents correcteurs K .

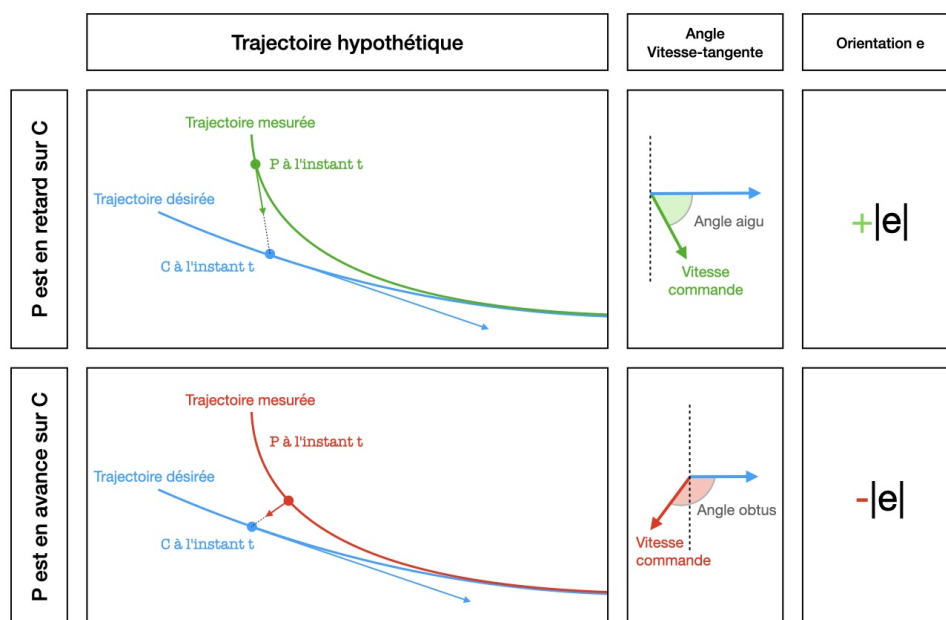


FIGURE 4.1 – Erreur signée

On peut maintenant rédiger les fonctions de suivi de trajectoire. On va donc boucler l'algorithme que l'on a décrit précédemment. On définit une fonction trajectoire qui renvoie le point C et la vitesse à l'instant t ("traj(t) -> [C(t), v(t)]"). On donne comme paramètres :

- la durée d'acquisition T
- N qui est le nombre de subdivisions de T que l'on souhaite. On a alors $dt = T/(N-1)$. Plus N est grand, plus le dt est petit et plus les modèles convergent (voir annexe G)
- et K notre gain de commande.

Lors de l'acquisition on stocke :

- les points de passage de P
- les angles q_1, q_2, q_3 (au cas où on souhaiterait refaire la géométrie directe, notamment pour connaître le déplacement individuel de chaque pièce)
- l'erreur et l'erreur signée.

On n'a pas besoin de stocker les points C car par définition la trajectoire à suivre est entièrement déterminée (en pratique on les stocke quand même car ils dépendent de notre T et N).

2^{ème} approximation : On ne modélisera pas le décalage du coude

$$d_2 = 0 \Leftrightarrow \overrightarrow{AB} = \overrightarrow{0}$$

3^{ème} approximation : On ne prend pas en compte les limites articulaires, on considère donc un workspace régulier.

On décide de modéliser 2 trajectoires, le cercle incliné et le carré dans le plan Oyz . Vous trouverez en annexe H les programmes pour les différentes trajectoires, les fonctions de calculs des modèles dont on aura besoin (modèle de géométrie direct, de cinématique inverse, de jacobienne), la fonction d'acquisition et la fonction qui trace les graphiques qui vont suivre.

Rappel : $v_{cmd} = v_{traj} + Ke \Leftrightarrow$ Le gain K intervient directement dans la loi de commande. Son influence est étudiée à travers deux situations différentes. Dans un premier temps, le robot est placé exactement sur la trajectoire au début de la simulation afin d'évaluer sa capacité à la conserver. Dans un second temps (partie suivante), le robot partira d'une position différente afin d'étudier sa capacité à rejoindre rapidement la trajectoire désirée.

4.3.1 Même condition initiale : capacité à rester sur la trajectoire

Pour cette première étude, on impose les mêmes conditions initiales pour toutes les simulations :

$$P(t = 0) = C(t = 0).$$

L'objectif n'est donc pas de rejoindre la trajectoire mais uniquement d'évaluer la capacité du robot à la suivre au cours du temps.

Cercle incliné

On commence par étudier le cercle incliné présenté précédemment. Cette trajectoire est régulière puisque sa courbure est constante et qu'elle ne présente aucune discontinuité de direction.

La figure 4.2 présente le résultat obtenu pour un coefficient de gain $K = 1$. On observe déjà une très bonne superposition entre la trajectoire désirée et la trajectoire mesurée. Un léger écart reste visible localement, mais celui-ci demeure faible.

Lorsque le gain K augmente, cet écart diminue progressivement. À partir de $K = 10$, les deux trajectoires deviennent pratiquement confondues à l'échelle de la figure et aucune amélioration visuelle significative n'est observée pour des gains encore plus élevés. Cette observation est confirmée quantitativement par la figure 4.3, qui représente l'évolution de l'erreur signée au cours du temps pour différentes valeurs de K .

Même pour $K = 1$, l'erreur reste inférieure au centimètre et demeure de l'ordre de quelques millimètres pendant la majeure partie du mouvement. L'augmentation du gain améliore encore légèrement la précision, mais le bénéfice devient rapidement négligeable.

Suivi de trajectoire, $K=1$, $N=1000$

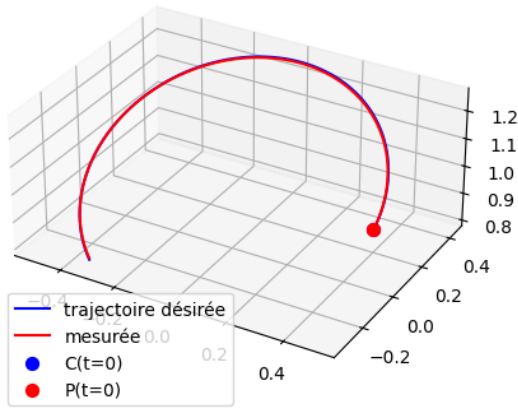


FIGURE 4.2 – graphe suivi de trajectoire

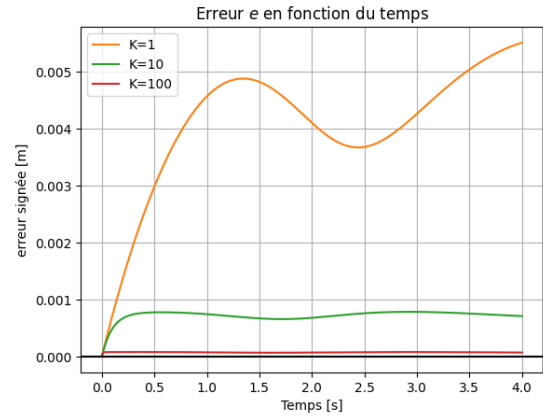


FIGURE 4.3 – comparaison de K

En revanche, un coefficient de gain trop important dégrade fortement le comportement du système. L'effecteur n'épouse plus correctement la trajectoire désirée mais oscille continuellement autour de celle-ci. Ces oscillations sont mieux visibles sur l'agrandissement présenté figure 4.4. L'évolution de l'erreur signée en figure 4.5 confirme ce comportement.

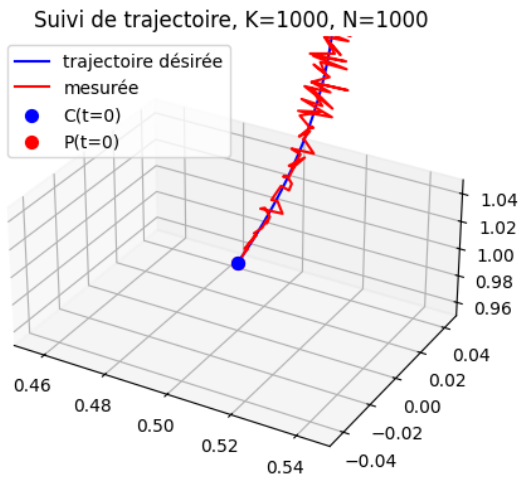


FIGURE 4.4 – Vue zoomée de la trajectoire

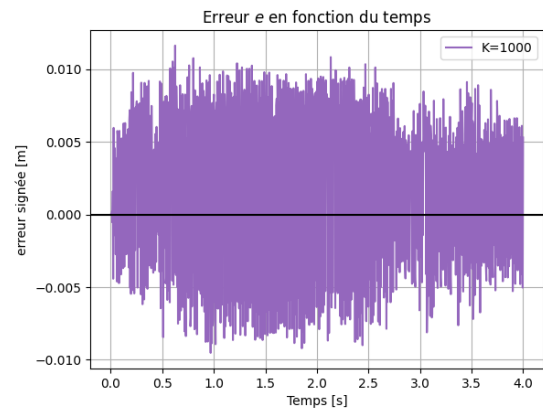


FIGURE 4.5 – Pour $K=1000$ le comportement du système se dégrade fortement

L'erreur change continuellement de signe, traduisant le fait que l'effecteur dépasse sans cesse la trajectoire désirée avant d'être ramené dans l'autre sens par une correction trop importante. Le système devient alors très oscillant sans pour autant améliorer la précision moyenne du suivi.

On retiendra donc que, pour une trajectoire régulière comme le cercle, un gain relativement faible suffit déjà à obtenir un excellent suivi. Augmenter davantage K n'apporte pratiquement aucun bénéfice et finit même par dégrader la stabilité du mouvement.

Trajectoire carrée

Afin de mettre davantage en évidence l'influence du gain, on considère maintenant une trajectoire plus difficile à suivre : un carré situé dans le plan Oyz . Contrairement au

cerle, cette trajectoire présente des changements brusques de direction aux sommets. À chaque angle, le vecteur vitesse change instantanément d'orientation, ce qui constitue un cas beaucoup plus exigeant pour l'algorithme de suivi.

Les simulations montrent que, quelle que soit la valeur de K , le robot présente une divergence initiale avant de converger progressivement vers la trajectoire désirée. Il semble donc impossible, avec cette loi de commande, de supprimer complètement cette phase transitoire.

Pour des valeurs faibles du gain, le robot est incapable de rattraper rapidement la trajectoire. Avec $K = 3$, les trajectoires désirée et mesurée ne deviennent pratiquement confondues qu'au début du troisième côté du carré. En augmentant le gain à $K = 5$, la convergence intervient plus tôt, vers la fin du deuxième côté. Lorsque le coefficient atteint des valeurs comprises entre 10 et 100, le comportement évolue sensiblement. La divergence initiale reste importante, mais le robot corrige beaucoup plus rapidement son erreur et rejoint la trajectoire dès le premier côté du carré. Voir annexe I pour les hodographe respectifs.

Cette amélioration apparaît clairement sur la comparaison des erreurs signées pour différentes valeurs de K .

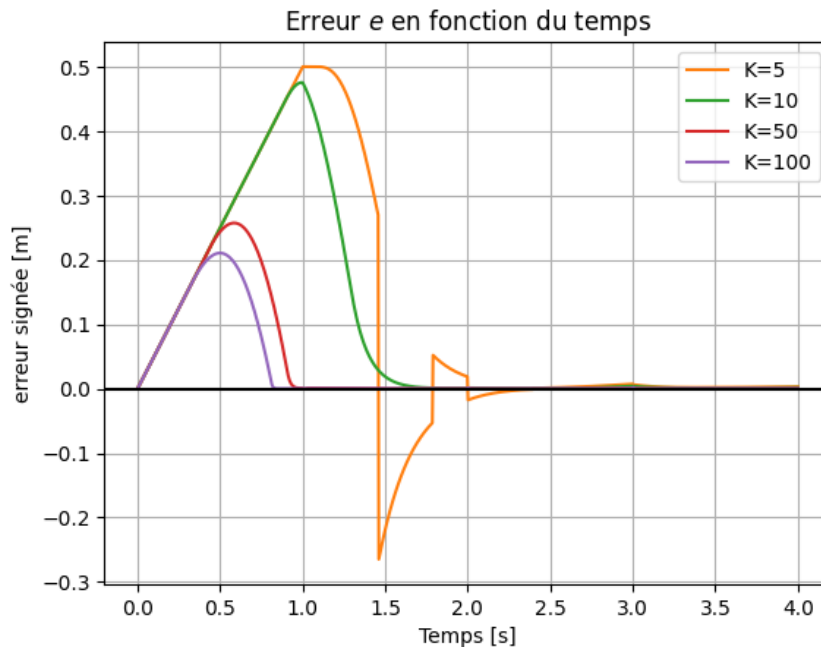


FIGURE 4.6 – Comparaison des K pour le carré plan lorsque $P_0 = C_0$

On remarque notamment que, bien que la divergence initiale paraisse plus marquée sur les hodographes pour les valeurs élevées de K , c'est finalement le cas $K = 5$ qui présente à la fois l'erreur maximale la plus importante et la durée de convergence la plus longue.

Enfin, comme pour la trajectoire circulaire, un coefficient trop élevé devient contre-productif. Pour $K = 1000$, les oscillations réapparaissent tout au long de la trajectoire (voir hodographe en annexe I). L'augmentation du gain ne permet pas de rejoindre plus rapidement la trajectoire désirée mais introduit des oscillations permanentes de l'effecteur. Là encore, une valeur excessive de K ne présente donc aucun intérêt pratique.

Cette situation nous permet aussi de constater les limites du modèle de cinématique inverse. Pour en savoir plus, lire la deuxième partie de l'annexe G.

4.3.2 Conditions initiales différentes : capacité à rattraper une trajectoire

On considère maintenant le cas où le robot ne démarre plus sur la trajectoire :

$$P(t = 0) \neq C(t = 0).$$

L'objectif est alors de rejoindre la trajectoire désirée le plus rapidement possible.

En partant de l'extérieur du cercle

La figure 4.7 compare l'erreur signée obtenue pour plusieurs valeurs de K . Entre $K = 1$ et $K = 10$, la convergence devient de plus en plus rapide. Les hodographes (voir annexe J) montrent également que la trajectoire mesurée devient tangente à la trajectoire désirée de plus en plus tôt.

Cette amélioration provient de la loi de commande

$$v_{\text{cmd}} = v_{\text{traj}} + K e,$$

mais également de l'intégration des vitesses articulaires :

$$q_i(t) = q_i(0) + \int_0^t \dot{q}_i(\tau) d\tau,$$

qui est approchée numériquement par

$$q_i(t + dt) = q_i(t) + \dot{q}_i dt.$$

Les corrections successives s'accumulent donc au cours du temps, ce qui permet au robot de rejoindre progressivement la trajectoire.

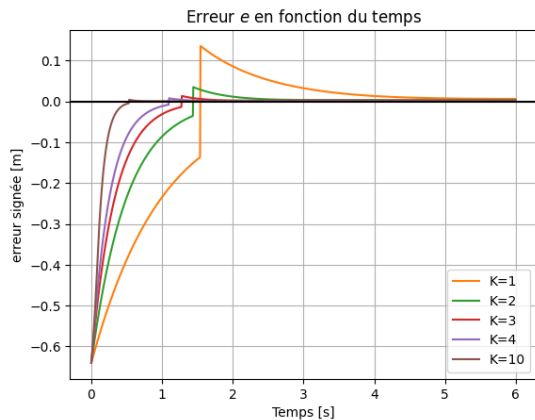


FIGURE 4.7 – –

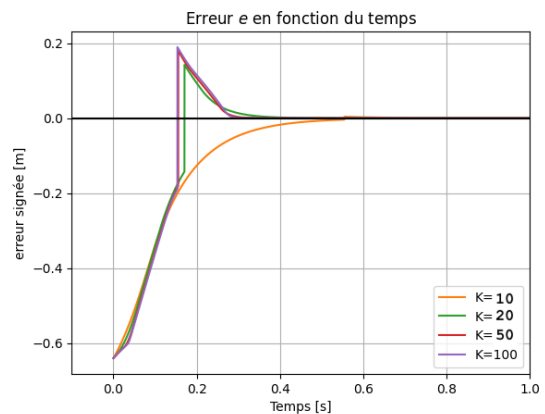


FIGURE 4.8 – –

En revanche, augmenter davantage le gain ne présente plus d'intérêt. Pour $K = 20, 50$ ou 100 , les corrections deviennent beaucoup plus brusques. Comme le montre la figure 4.8, l'erreur effectue un saut important dès le début du mouvement ; pour $K = 100$, elle passe d'environ $-0,6$ m à $0,2$ m en moins de $0,2$ s (une vue de $K = 100$ uniquement est disponible en annexe K). Le robot dépasse alors rapidement la trajectoire au lieu de la rejoindre progressivement.

On retiendra donc qu'un gain élevé permet de converger rapidement mais pas nécessairement de mieux converger. Les performances restent de toute façon limitées par les vitesses maximales des articulations.

En partant de l'intérieur du cercle

Lorsque le robot démarre à l'intérieur du cercle, la trajectoire mesurée rejoint progressivement la trajectoire désirée en décrivant une spirale.

Pour tous les gains K , l'effecteur finit par dépasser ponctuellement le point cible C , ce qui se traduit par un changement de signe de l'erreur signée. Cela ne remet toutefois pas en cause la convergence puisque la norme de l'erreur continue de diminuer jusqu'à tendre vers zéro.

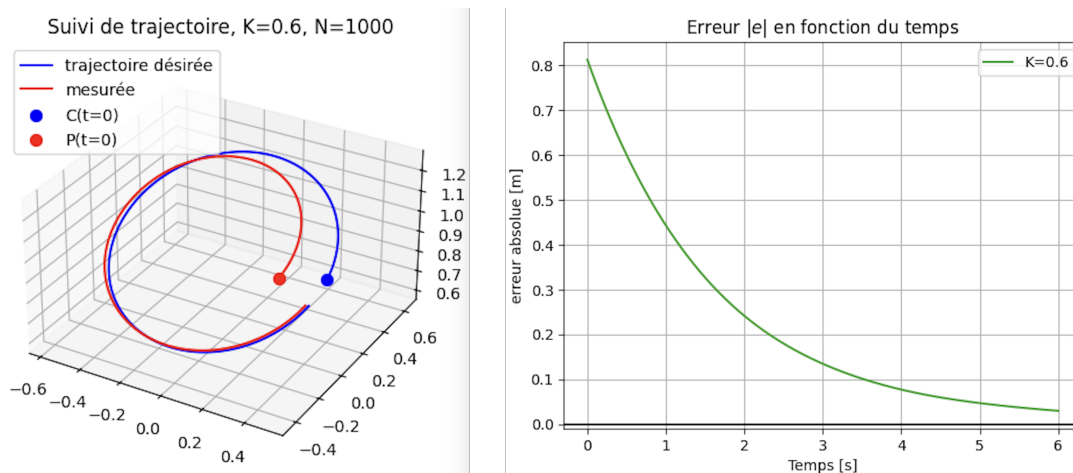


FIGURE 4.9 – La "spirale", un exemple avec $K=0,6$

En conclusion, le choix du coefficient de gain résulte d'un compromis entre rapidité de convergence et stabilité. Un gain trop faible conduit à un suivi lent et imprécis, tandis qu'un gain trop élevé engendre des oscillations importantes sans améliorer significativement les performances. Dans notre étude, des valeurs comprises entre 10 et 100 offrent le meilleur compromis pour les trajectoires considérées.

Conclusion

Au cours de ce stage, nous avons développé un modèle simplifié d'un robot manipulateur de type PUMA en abordant successivement sa géométrie, son espace de travail puis son suivi de trajectoire. Après avoir établi les modèles de géométrie directe et inverse, nous avons introduit la cinématique différentielle ainsi qu'une loi de commande reposant sur la matrice jacobienne et sa pseudo-inverse. Les simulations réalisées montrent que cette approche permet d'obtenir un suivi de trajectoire précis, tout en mettant en évidence l'influence du gain de commande sur la stabilité et la rapidité de convergence.

Ce travail constitue une première étape vers une modélisation plus réaliste. Une suite naturelle consisterait à introduire la dynamique du robot en prenant en compte les masses des différents solides, leurs inerties ainsi que les efforts transmis dans les liaisons. Il serait alors possible de déterminer les couples moteurs nécessaires au mouvement, d'étudier l'influence des accélérations et des charges transportées, ou encore de mettre en œuvre des lois de commande plus élaborées intégrant directement les effets dynamiques du système.

N'hésitez pas à consulter notre [Github](#) et notamment le fichier du compte rendu `cr_stage.ipynb` qui est un notebook Python permettant d'exécuter (et de tester) tous les programmes que vous trouverez en annexe.

Bibliographie et sources

Bibliographie

1. Juste Un Petit Cours. (2021, 4 février). *Méthode de Denavit Hartenberg* [Vidéo]. YouTube. <https://www.youtube.com/watch?v=9BmlFr0jAJg>
2. ResearchGate. (s. d.). *Discontinuous part of torque for one joint variable*. https://www.researchgate.net/figure/Discontinuous-part-of-torque-for-one-joint-variable_fig2_264084551
3. Robotics, Back-End. (2022). *Inverse Kinematics Explained Simply* [Vidéo]. YouTube. <https://www.youtube.com/watch?v=6K0EQfsgy8Q>
4. The Construct. (2020). *Denavit-Hartenberg Parameters Explained* [Vidéo]. YouTube. <https://www.youtube.com/watch?v=Ta8cKqltPfU>
5. EngineerJau. (2013, 15 juillet). *On the Basis of Workspaces of Robotic Manipulators – Part 2*. <https://engineerjau.wordpress.com/2013/07/15/on-the-basis-of-workspaces-part-2/>
6. Robotics Stack Exchange. (2017). *Modified DH Parameters for PUMA 560 – Wrong Results*. <https://robotics.stackexchange.com/questions/11887/modified-dh-parameters-for-puma-560-wrong-results>
7. Juecoree. (2022). *Forward Kinematics of PUMA 560 Robot Using DH Method*. Hive Blog. <https://hive.blog/hive-196387/@juecoree/forward-kinematics-of-puma-560-robot-using-dh-method>

Sources

1. **Figure 1.1** : Cults3D. Image du modèle 3D, sans légende, utilisée comme illustration.
2. **Figure 1.2** : Carter, H. V., & Gray, H. (1918). *Gray's Anatomy : Anatomy of the Human Body*, planche 1231. Bartleby.com. Image modifiée à l'aide d'une intelligence artificielle.
3. **Figure 3.3** Scribd. (s. d.). *R. Merzouki – Modélisation Robot PUMA*. <https://fr.scribd.com/document/618591054/Emailing-modelisation-Robot-PUMA>

A Workspace modèle simplifié (en 2D)

Initialisation

```
1 # position cible
2 x = 3
3 y = 3
4
5 # Parametres
6 L1 = 3
7 L2 = 3
8
9 # Limites articulaires
10 I_q1 = (-3*np.pi/4, 3*np.pi/4)
11 I_q2 = (-np.pi, np.pi)

1 # modeles de geometrie inverse
2 def q1(x,y) :
3     return np.arctan(y/x) - np.arccos( (L1**2 - L1**2 + x**2+ y**2)
4     / (2*L1*np.sqrt(x**2 + y**2)) )
5
6 def q2(x,y) :
7     return np.pi - np.arccos( (L1**2 + L2**2 - x**2 - y**2) / (2*L1
8     *L2) )
```

Workspace et quelques tests

```
1 def q(x,y):
2     """ Entree : coordonnees (x,y)
3     Sortie : [q1,q2] si le point est atteignable
4             False sinon"""
5
6     r = np.sqrt(x**2 + y**2)
7     if abs(L1-L2) <= r <= L1+L2:
8         q_1 = q1(x,y)
9         q_2 = q2(x,y)
10        if I_q1[0] <= q_1 <= I_q1[1] and I_q2[0] <= q_2 <= I_q2[1]:
11            return [float(q_1), float(q_2)]
12        else:
13            print("pas de solution")
14            return False

1 x,y = 4,0
2 q(x,y)
```

[-0.8410686705679303, 1.6821373411358604]

On essaye ensuite de tracer le workspace mais on a plusieurs problèmes :

1. les fonctions qu'on utilise ne sont pas vectorisées donc on peut pas utiliser la méthode de "mask" qu'on a vu précédemment
2. on divise parfois par 0 dans la fonction arctan

En utilisant la fonction arctangente 2 de Numpy, on corrige le problème numéro 2 (la fonction `atan2(y,x)` conserve l'information de quadrant et évite les problèmes liés à

la division par zéro contrairement à atan). On propose donc ces fonctions révisées qui servent à tracer les workspace qui sont dans le rapport :

```

1 # modeles de geometrie inverse (vectorises)
2 def q1(x, y):
3     r = np.sqrt(x**2 + y**2)
4     with np.errstate(divide='ignore', invalid='ignore'):
5         angle = np.arctan2(y, x) - np.arccos((L1**2 - L2**2 + x**2
6 + y**2) / (2 * L1 * r))
7     return angle
8
9 def q2(x, y):
10     r_squared = x**2 + y**2
11     with np.errstate(divide='ignore', invalid='ignore'):
12         angle = np.pi - np.arccos((L1**2 + L2**2 - r_squared) / (2
13 * L1 * L2))
14     return angle
15
16 # Filtrage des positions
17 def q(x, y):
18     """ Retourne un masque boolean des points (x,y) atteignables et
19 dans les limites articulaires."""
20
21     r = np.sqrt(x**2 + y**2)
22     reachable = (np.abs(L1 - L2) <= r) & (r <= L1 + L2)
23     q1_vals = q1(x, y)
24     q2_vals = q2(x, y)
25     q1_valid = (I_q1[0] <= q1_vals) & (q1_vals <= I_q1[1])
26     q2_valid = (I_q2[0] <= q2_vals) & (q2_vals <= I_q2[1])
27     return reachable & q1_valid & q2_valid
28
29 # dessiner le workspace
30 N = 500
31 x = np.linspace(-(L1 + L2), L1 + L2, N)
32 y = np.linspace(-(L1 + L2), L1 + L2, N)
33 X, Y = np.meshgrid(x, y)
34
35 mask = q(X, Y)
36
37 fig, ax = plt.subplots()
38 ax.scatter(X[mask], Y[mask], s=1)
39
40 ax.set_aspect('equal')
41 plt.grid(True)
42
43 ax.set_title(f'Workspace modele simple \n L1, L2 = ({L1},{L2}) \n (
44 $q_1$, $q_2$) \n in [{I_q1[0]:.2f} ; {I_q1[1]:.2f}] x [{I_q2
45 [0]:.2f} ; {I_q2[1]:.2f}]', size='11')
46
47 ax.set_xlabel('x')
48 ax.set_ylabel('y')
49
50 plt.show()

```

B Modèle de géométrie directe (MGD)

```
1 # Valeurs (proportionnelles entre elles)
2 d1 = 66.0 # Hauteur de la base (Socle -> Axe 2)
3 d2 = 20.0 # Decalage de l'épaule (Axe 1 -> Axe 2)
4 a2 = 43.2 # Longueur de l'arriere bras (Axe 2 -> Axe 3)
5 a4 = 43.2 # Longueur de l'avant-bras (Axe 3 -> Centre outil)
6
7 # Fonctions MGD
8 def OA():
9     return np.array([0.0, 0.0, d1])
10
11 def OB(q1):
12     q1 = np.atleast_1d(q1)
13     x = -d2 * np.sin(q1)
14     y = d2 * np.cos(q1)
15     z = np.full_like(q1, d1)
16     return np.vstack((x, y, z)).squeeze()
17
18 def OC(q1, q2):
19     q1 = np.atleast_1d(q1)
20     q2 = np.atleast_1d(q2)
21     x = -d2 * np.sin(q1) + a2 * np.cos(q2) * np.cos(q1)
22     y = d2 * np.cos(q1) + a2 * np.cos(q2) * np.sin(q1)
23     z = d1 + a2 * np.sin(q2)
24     z = np.broadcast_to(z, q1.shape)
25     return np.vstack((x, y, z)).squeeze()
26
27 def OD(q1, q2, q3):
28     q1 = np.atleast_1d(q1)
29     q2 = np.atleast_1d(q2)
30     q3 = np.atleast_1d(q3)
31     x = -a4 * np.sin(q2 + q3) * np.cos(q1) - d2 * np.sin(q1) + a2 *
32     np.cos(q2) * np.cos(q1)
33     y = -a4 * np.sin(q2 + q3) * np.sin(q1) + d2 * np.cos(q1) + a2 *
34     np.cos(q2) * np.sin(q1)
35     z = d1 + a2 * np.sin(q2) + a4 * np.cos(q3 + q2)
36     z = np.broadcast_to(z, q1.shape)
37     return np.vstack((x, y, z)).squeeze()
38
39 #schema 3d
40 fig = plt.figure()
41 ax = fig.add_subplot(111, projection='3d')
42
43 q1=.0
44 q2=0
45 q3=0
46
47 A = OA()
48 B = OB(q1)
49 C = OC(q1, q2)
50 D = OD(q1, q2, q3)
```

```
50 X = [0,A[0], B[0], C[0], D[0]]
51 Y = [0,A[1], B[1], C[1], D[1]]
52 Z = [0,A[2], B[2], C[2], D[2]]
53
54 ax.plot(X, Y, Z, color="C1")
55 ax.scatter(X, Y, Z, color="C2")
56
57 R=100
58 ax.set_xlim(-R,R)
59 ax.set_ylim(-R,R)
60 ax.set_zlim(0,R)
61 ax.set_aspect('equal')
62
63 plt.show()
```

C Fonction de géométrie inverse

```
1 def q(x, y, z, s1=1, s3=1):
2     """ Calcule le modele geometrique inverse: position (x,y,z) ->
3     angles (q1,q2,q3)
4
5     s1 et s2 permettent de choisir la configuration articulaire :
6     config bras droit, bras gauche.
7     s1 et s3 doivent valoir 1 ou -1. Pre-remplis sur 1 (config
8     Righty) """
9
10    r_q1_sq = x**2 + y**2 - d2**2
11    if r_q1_sq < 0:
12        print("Erreur : x, y hors de portee")
13        return False
14
15    q1 = np.arctan2(y, x) - np.arctan2(d2, s1 * np.sqrt(r_q1_sq))
16
17    R = x * np.cos(q1) + y * np.sin(q1)
18    Z = z - d1
19
20    D = (a2**2 + a4**2 - (R**2 + Z**2)) / (2 * a2 * a4)
21
22    if abs(D) > 1:
23        print("Erreur : Distance hors de portee")
24        return False
25
26    q3 = np.arctan2(D, s3 * np.sqrt(1 - D**2))
27
28    A = a2 - a4 * np.sin(q3)
29    B = a4 * np.cos(q3)
30
31    q2 = np.arctan2(A * Z - B * R, A * R + B * Z)
32
33    if not (np.deg2rad(I_q1[0]) <= q1 <= np.deg2rad(I_q1[1])) or \
34    not (np.deg2rad(I_q2[0]) <= q2 <= np.deg2rad(I_q2[1])) or \
35    not (np.deg2rad(I_q3[0]) <= q3 <= np.deg2rad(I_q3[1])):
36        print("Erreur : Limites articulaires depassees")
37        return False
38
39    return q1, q2, q3
```

D Acquisition des points du workspace (méthode par échantillonnage)

```
1 # acquisition des points
2
3 N_q1, N_q2, N_q3 = 34, 34, 34
4
5 q1_values = np.linspace(I_q1[0], I_q1[1], N_q1) # q1_min, q1_max,
6         nombre d'échantillons par articulation
7 q2_values = np.linspace(I_q2[0], I_q2[1], N_q2)
8 q3_values = np.linspace(I_q3[0], I_q3[1], N_q3)
9
10 workspace_points = []
11
12 for q1 in q1_values:
13     for q2 in q2_values:
14         for q3 in q3_values:
15             # Matrices DH
16             T_1 = T_i(d1, q1, 0, -np.pi/2)
17             T_2 = T_i(d2, q2, a2, 0)
18             T_3 = T_i(d3, q3, a4, 0)
19
20             # Transformation totale de 0 a 3 (par chaine)
21             T_03 = T_1 @ T_2 @ T_3
22
23             # Position (x, y, z) du bout du bras
24             P = T_03[:3, 3]
25
26             workspace_points.append(P)
27
28 workspace_points = np.array(workspace_points)
```

Et pour tracer les nuages de points :

```
1 fig = plt.figure()
2 ax = fig.add_subplot(111, projection="3d")
3
4 ax.scatter(
5     workspace_points[:, 0],
6     workspace_points[:, 1],
7     workspace_points[:, 2],
8     s=1, color='C3'
9 )
10
11 ax.set_xlabel("x")
12 ax.set_ylabel("y")
13 ax.set_zlabel("z")
14 ax.set_title(f"Workspace obtenu par echantillonnage", size=10)
15 ax.set_aspect('equal')
16 plt.show()
```

résultats : voir figure 3.5

E Résultats d'acquisition du workspace

```
1 # enregistrement des points
2 with open("workspace.txt", "w") as f:
3     for P in workspace_points:
4         f.write(f"{P[0]} {P[1]} {P[2]}\n")
5
6 # comptage des lignes du programme precedent (cad du nombre de
   points)
7 with open("workspace.txt", "r") as f:
8     nb_lignes = sum(1 for _ in f)
9
10 print("Nombre de points :", nb_lignes)
```

Nombre de points : 39304

Les 20 premières lignes du fichier workspace.txt :

```
0.03396799840272725 -0.0034102648263273838 0.07866041976687087
0.028171064616649207 -0.005536779636180805 0.07825541966600774
0.022487524055140273 -0.00762169790891966 0.07697439893234238
0.01703345061570302 -0.00962243974434406 0.07484351961534533
0.01192023183125453 -0.011498144362061773 0.07190630026912112
0.007252294023146973 -0.013210504593647449 0.06822272718102826
0.003124969621724226 -0.014724549222857002 0.06386802928282909
-0.0003774497902677386 -0.016009357196307736 0.05893114176413297
-0.003183434949082336 -0.01703868911843154 0.05351288976547074
-0.005235679738893866 -0.01779152313379664 0.04772392924510105
-0.006492271544000054 -0.018252484252572268 0.04168248707284489
-0.006927547222419266 -0.018412158351101805 0.03551194650459371
-0.006532617219491787 -0.018267284434808736 0.029338327348898748
-0.005315547117610212 -0.01782082123688562 0.023287712287733114
-0.003301192914322258 -0.01708188679263343 0.01748367191320159
-0.0005306933928909121 -0.016065572223513343 0.012044741068197109
0.0029393700474659827 -0.014792633533964806 0.00708199803123186
0.0070381289430251 -0.013289067715366448 0.0026967959853033234
0.011681875099082455 -0.01158558181429031 -0.0010213068994129621
0.01677577014692882 -0.009716965808170133 -0.003996376495505783
[...]
```

Le fichier entier est disponible sur github [ici](#).

Fonction "est dans le workspace" (rédigée par IA) :

```
1 from scipy.spatial import KDTree
2
3 # Construction de l'arbre a partir du workspace deja genere
4 tree = KDTree(workspace_points)
5
6 def est_dans_workspace(P_cible, epsilon=0.05):
7     """ Verifie si un point P_cible = [x, y, z]
8         appartient approximativement au workspace echantillonne """
9
10    P_cible = np.array(P_cible)
11
12    d_min, index = tree.query(P_cible)
13
14    return d_min < epsilon, d_min, workspace_points[index]
15
16
```

```

17 # Exemple d'utilisation
18 P_test = [0.3, 0.2, 0.4]
19
20 atteignable, distance, point_proche = est_dans_workspace(P_test,
21     epsilon=0.05)
22
23 if atteignable:
24     print("Point dans le workspace")
25 else:
26     print("Point hors du workspace")
27
28 print("Distance minimale :", distance)
29 print("Point du workspace le plus proche :", point_proche)

```

Résultat :

Point hors du workspace

Distance minimale : 0.40373372009458364

Point du workspace le plus proche : [0.05262284 0.03621515 0.12617528]

Avec :

```

1 P_test = [0.03, 0.02, 0.04]

```

Point dans le workspace

Distance minimale : 0.011775037836711733

Point du workspace le plus proche : [0.03784457 0.02664645 0.03426076]

F Suivi de trajectoire (instantané)

L'objectif de ce programme est simplement de voir à quoi peut ressembler la boucle que l'on construira par la suite... *(Il faut ajouter au dessus de ce programme la première cellule de l'annexe H)

```

1 # point de depart
2 x, y, z = 100, 0, 100
3
4 # point a atteindre
5 x_cible, y_cible, z_cible = 110, 0, 100
6
7 v_x = 10 # en m/s du coup
8 v_y = 0
9 v_z = 0
10
11 dt = 0.01 # en sec
12
13 # geom. indirecte pour obtenir les q_i
14 q1, q2, q3 = MGI(x, y, z)
15
16 # cinématique inverse : passer de v a q_point
17 qdot = cinématique_inv(q1,q2,q3, v_x,v_y,v_z)
18
19 #nouveau point = q + dq avec dq = q_point * dt
20 q1 = q1 + qdot[0] * dt
21 q2 = q2 + qdot[1] * dt
22 q3 = q3 + qdot[2] * dt
23
24 # geometrie inverse pour (x,y,z)
25 x,y,z = MGD(q1,q2,q3)
26
27 print(f"x: {x:.2f} y: {y:.2f} z: {z:.2f}")

```

Pour passer à l'étape suivante :

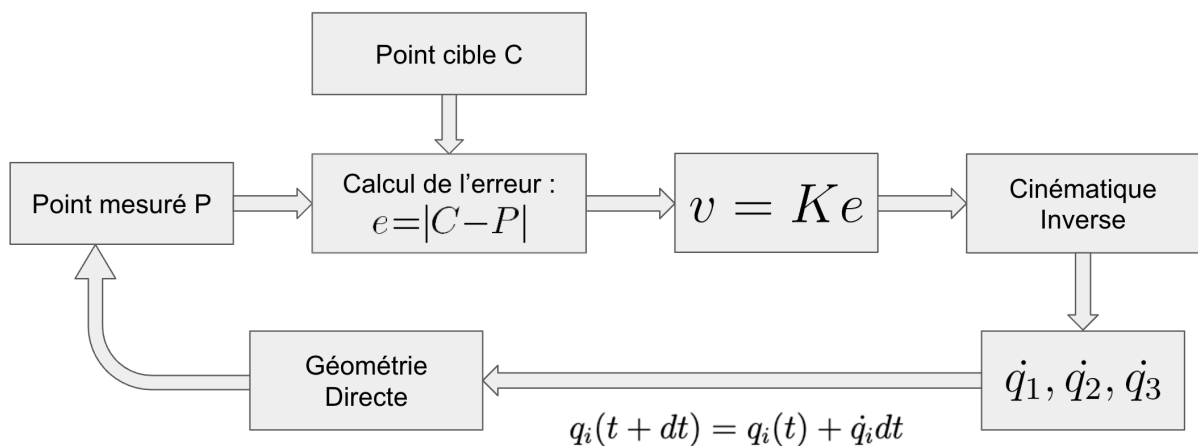


FIGURE F.1 – Schéma boucle/conceptuel de l'algorithme suivi de trajectoire

G Suivi de trajectoire : preuve de la convergence en petit dt

Avec le programme précédent on veut atteindre le point C(110, 0, 100) depuis le point de départ P(100, 0, 100) à la vitesse $v = (10, 0, 0)$ (c'est-à-dire 10m/s dans la direction x). Après un $dt = 1s$ on devrait s'y trouver car $10 \text{ m/s} \times 1 \text{ s} = 10 \text{ m}$. Or en executant le code on obtient : $x: 109.29 \ y: 0.00 \ z: 99.53$

Si on refais la même chose un facteur de 10 en dessous : $dt = 0.1 \text{ s}$, on doit être en C(101, 0, 100) après execution. Le code renvoie : $x: 100.99 \ y: 0.00 \ z: 100.00$
On a toujours pas convergé mais on est beaucoup plus proche.

Encore, cette fois avec $dt = 0.01$, on doit être en C(100.1, 0, 100). Le programme renvoie : $x: 100.10 \ y: 0.00 \ z: 100.00$

On observe donc une amélioration significative entre un dt de l'ordre de 10^0 et de l'ordre de 10^{-2} . On doit donc prendre un N très grand pour avoir des valeurs très précises.

Pour aller plus loin : Limites de notre modèle

Puisque c'est lié avec ce que l'on vient de dire, nous mettons cette partie ici. Il est préférable d'avoir lu la partie 4.3.1 avant.

On remarque que la trajectoire mesurée (dans la situation avec le carré et en ayant les mêmes conditions initiales) diverge, certes, de la trajectoire désirée mais surtout diverge hors du plan du carré! Or si on s'en tient à notre algorithme, e le vecteur d'erreur est censé être orienté uniquement selon y .

Que se passe t'il ? On part de $P_0 = C_0$. D'après notre algorithme, on commence par calculer l'erreur puis la vitesse à appliquer avec la formule $v = K \times e$ et on en déduit les vitesse articulaires puis la nouvelle position du point P. Or en théorie, P_{0+dt} doit être égal à P_0 puisque l'erreur à l'instant initial est nulle (points confondus). Donc si on dit que

$$P_0 = C_0 = \begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix}$$

Alors

$$P_{0+dt} = \begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix}$$

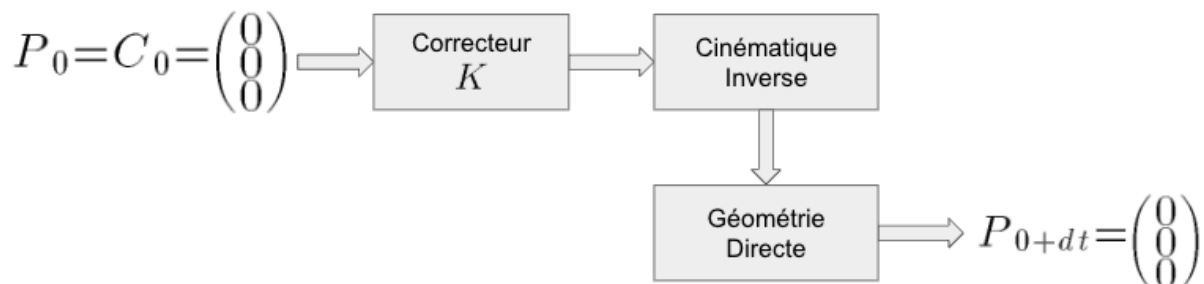


FIGURE G.1 – En théorie

Sauf que comme on vient de le voir, les modèles de géométrie et de cinématique (et en particulier les modèles inverses) introduisent une erreur dans le résultat. Ainsi un $x = 0$ est retransmis comme $x = 0.0001$ ce qui est proche mais suffit à détourner e d'une direction purement selon y :

$$P_{0+dt} = \begin{pmatrix} 0,001 \\ 0,001 \\ -0,0001 \end{pmatrix}$$

Lorsque K est très grand (comme $K = 1000$) celui ci agit comme multiplicateur de l'approximation introduite par les modèles et "dt après dt", la trajectoire commence à diverger fortement dans l'espace xyz . D'où le faite que la trajectoire mesuré quitte le plan du carré.

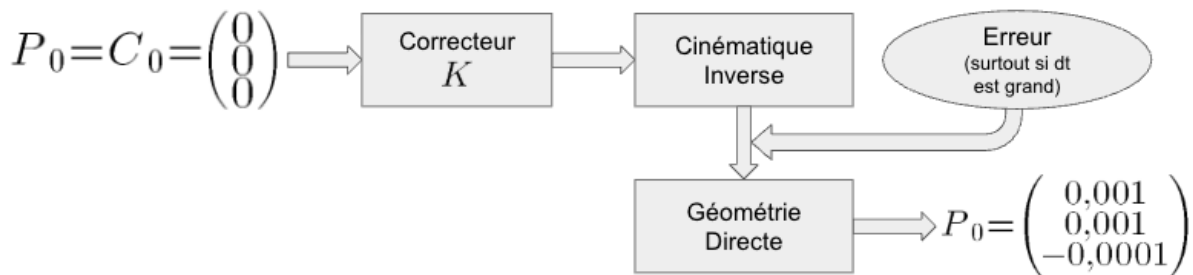


FIGURE G.2 – En pratique

Pour s'en convaincre voici un exemple avec la trajectoire carré, une durée d'acquisition $t = 4$ les mêmes conditions initiales, $K = 1000$ (grand multiplicateur) et $N=1\ 000$ puis $N=10\ 000$. On a alors un dt plus faible dans la deuxième situation (0,004 et 0,0004 respectivement) ce qui confirme la convergence en petit dt .

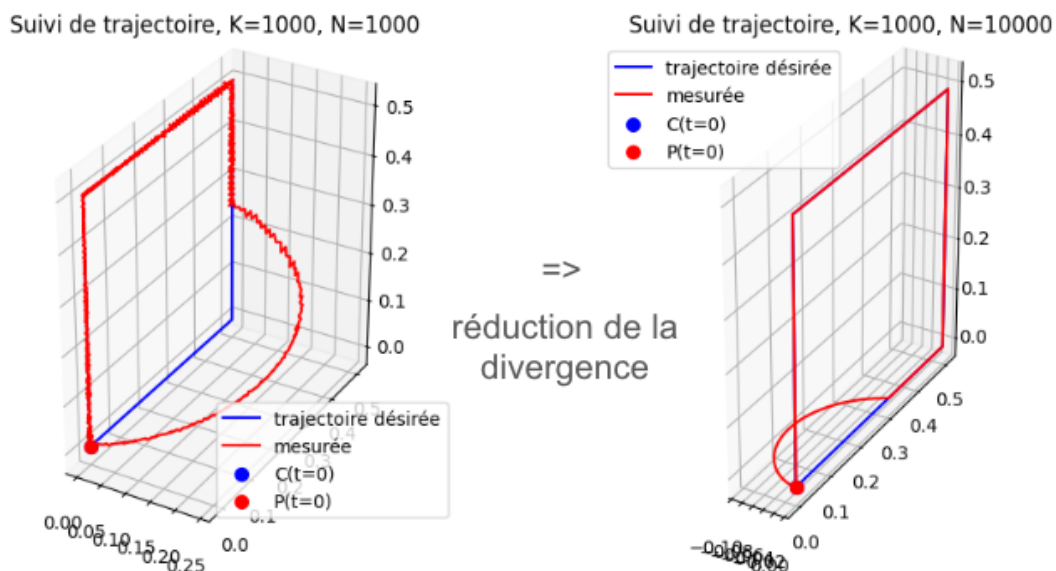


FIGURE G.3 – Comparaison

H Suivi de trajectoire (code complet)

Fonctions de calculs des modèles

```
1 # modele de geometrie directe
2
3 def MGD(q1, q2, q3):
4     """Retourne (x,y,z)
5     Angles en radians"""
6
7     r = a1*np.cos(q2) + d4*np.cos(q2 + q3)
8     x = r*np.cos(q1)
9     y = r*np.sin(q1)
10    z = d1 + a1*np.sin(q2) + d4*np.sin(q2 + q3)
11
12    return x, y, z
13
14 # modele de geometrie inverse
15 # (le meme que precedement mais sans les tests)
16
17 def MGI(x, y, z, s3=1):
18     """Calcule le modele geometrique inverse: position (x,y,z) ->
19     angles (q1,q2,q3).
20     s3 = +1 ou -1 choisit la config bras gauche, bras droit"""
21
22    R = np.sqrt(x**2 + y**2)
23    Z = z - d1
24    q1 = np.arctan2(y, x)
25    c3 = (R**2 + Z**2 - a1**2 - d4**2) / (2*a1*d4)
26    c3 = np.clip(c3, -1.0, 1.0)
27    s3_val = s3 * np.sqrt(1 - c3**2)
28    q3 = np.arctan2(s3_val, c3)
29    q2 = np.arctan2(Z, R) - np.arctan2(d4*np.sin(q3), a1 + d4*np.
30    cos(q3))
31
32    return q1, q2, q3
33
34 # Jacobienne
35
36 def jacobienne(q1, q2, q3):
37     """ Retourne J(q)
38     Angles en radian """
39     # variables intermediaires
40    r = a1*np.cos(q2) + d4*np.cos(q2 + q3)
41    dr_dq2 = (-a1*np.sin(q2)-d4*np.sin(q2 + q3))
42    dr_dq3 = (-d4*np.sin(q2 + q3))
43
44    J = np.array([
45        [-r*np.sin(q1) , np.cos(q1)*dr_dq2 , np
46        .cos(q1)*dr_dq3],
47        [r*np.cos(q1) , np.sin(q1)*dr_dq2 , np
```

```

    .sin(q1)*dr_dq3],
47     [    0.0          ,  a1*np.cos(q2) + d4*np.cos(q2 + q3) ,  d4
    *np.cos(q2 + q3)]
48     ])
49
50     return J
51
52
53
54 # LA fonction ultime
55 # (pseudo-inverse amortie "Damped Least Squares")
56 # redigee avec l'IA (uniquement celle-ci)
57
58 def cinematique_inv(q1,q2,q3,  vx,vy,vz,  damping=0.05,qdot_max=np.
    inf) :
59     """
60     Cinematique inverse differentielle robuste.
61
62     Parametres
63
64     -----
65
66     q1,q2,q3 : float
67
68         Angles articulaires [rad]
69
70     vx,vy,vz : float
71
72         Vitesse cartesienne desiree
73
74     damping : float
75
76         Amortissement DLS (0.01 a 0.1 typiquement)
77
78     qdot_max : float
79
80         Saturation max des vitesses articulaires
81
82         (np.inf pour desactiver)
83
84     Retour
85
86     -----
87
88     qdot : ndarray(3,)
89
90         [q1_dot, q2_dot, q3_dot]
91
92     """
93
94     J = jacobienne(q1, q2, q3)
95
96     v = np.array([vx, vy, vz], dtype=float)

```

```

97
98     # Damped Least Squares
99
100    A = J @ J.T + (damping**2) * np.eye(3)
101
102    qdot = J.T @ np.linalg.solve(A, v)
103
104    # Saturation eventuelle
105
106    max_abs = np.max(np.abs(qdot))
107
108    if max_abs > qdot_max:
109
110        qdot *= qdot_max / max_abs
111
112    return qdot

```

Différentes trajectoires : le cercle incliné

```

1  # le cercle dans le plan incline
2  def traj(t, alpha_deg=30):
3      R = 1
4      alpha = np.radians(alpha_deg)
5
6      # Cercle a plat
7      x_f, y_f = R * np.cos(t), R * np.sin(t)
8      ux_f, uy_f = -R * np.sin(t), R * np.cos(t)
9
10     # Rotation autour de X + offset Z
11     x = x_f
12     y = y_f * np.cos(alpha)
13     z = y_f * np.sin(alpha) + 1
14
15     ux = ux_f
16     uy = uy_f * np.cos(alpha)
17     uz = uy_f * np.sin(alpha)
18
19     return x, y, z, ux, uy, uz

```

le carré dans le plan Oyz

```

1  # carre de cotes 1m dans le plan yOz
2  def traj(t) :
3      """ Definit la trajectoire
4      Parametre
5      ----
6      t : float, temps en secondes
7      ----
8      Sortie :
9
10     x,y,z : coordonnees en fct(t)
11     ux, uy, uz : coordonnees du vecteur oriente
12     """

```

```

13     R = 1
14     if t <= 1:
15         x, y, z = 0, t*R, 0
16         ux, uy, uz = 0, R, 0
17     elif t <= 2:
18         x, y, z = 0, R, (t-1)*R
19         ux, uy, uz = 0, 0, 100
20     elif t <= 3:
21         x, y, z = 0, R-(t-2)*R, R
22         ux, uy, uz = 0, -R, 0
23     else:
24         x, y, z = 0, 0, R-(t-3)*R
25         ux, uy, uz = 0, 0, -R
26
27     return x,y,z, ux, uy, uz

```

La fonction d'acquisition

```

1  # PARAMETRES
2  N = 1000
3  T = """modifiable par lutilisateur"""
4  dt = T/(N-1)
5  K = """modifiable par lutilisateur"""
6
7  # vitesses max des articulations (rad/s)
8  Q1DOT_MAX = np.deg2rad(180)
9  Q2DOT_MAX = np.deg2rad(180)
10 Q3DOT_MAX = np.deg2rad(180) # soit v = R*pi = pi m/s
11
12 # etat initial du robot
13 x0 = 0
14 y0 = 0
15 z0 = 0
16 q1, q2, q3 = MGI(x0, y0, z0)
17
18 # STOCKAGE (initialisation)
19 temps = []
20 x_traj, y_traj, z_traj = [],[],[] # trajectoire cible
21 x_puma, y_puma, z_puma= [],[],[] # robot
22 q1_hist, q2_hist, q3_hist = [],[],[] # angles
23 erreur = [] # erreurs ||e|| distance
24 erreur_signee = [] # erreurs ||e|| distance * signe(angle obtu/aigu
25 )
26 angle_aigu = [] # depassement
27
28 for t in np.linspace(0, T, N):
29     xt, yt, zt, ux, uy, uz = traj(t) # cible
30     xp, yp, zp = MGD(q1, q2, q3) # robot
31
32     # erreur
33

```

```

34     e = np.array([xt - xp, yt - yp, zt - zp])
35     norme_e = np.linalg.norm(e) # "distance"
36
37     # vitesse cible
38
39     v_traj = np.array([ux, uy, uz])
40
41     # loi de commande
42
43     v_cmd = v_traj + K*e
44
45     # cinématique inverse
46
47     qdot = cinématique_inv(
48         q1,
49         q2,
50         q3,
51         v_cmd[0],
52         v_cmd[1],
53         v_cmd[2]
54     )
55
56     # saturation moteurs (redige par IA)
57
58     qdot[0] = np.clip(qdot[0], -Q1DOT_MAX, Q1DOT_MAX)
59     qdot[1] = np.clip(qdot[1], -Q2DOT_MAX, Q2DOT_MAX)
60     qdot[2] = np.clip(qdot[2], -Q3DOT_MAX, Q3DOT_MAX)
61
62     # integration (somme infinie)
63
64     q1 += qdot[0]*dt
65     q2 += qdot[1]*dt
66     q3 += qdot[2]*dt
67
68     # signe de l'erreur
69     s = np.dot(v_traj, e)
70     erreur_signee.append(np.sign(s)*norme_e)
71
72     angle_aigu.append(s >= 0)
73
74     # stockage
75
76     temps.append(t)
77
78     x_traj.append(xt)
79     y_traj.append(yt)
80     z_traj.append(zt)
81
82     x_puma.append(xp)
83     y_puma.append(yp)
84     z_puma.append(zp)
85
86     q1_hist.append(q1)

```

```

87     q2_hist.append(q2)
88     q3_hist.append(q3)
89
90     erreur.append(norme_e)

```

La fonction qui trace les graphiques

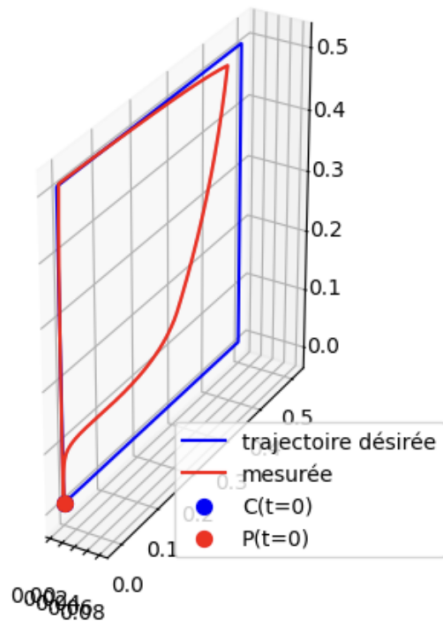
```

1  fig = plt.figure()
2  ax = fig.add_subplot(111, projection='3d')
3
4  # hodographes
5  ax.plot(x_traj, y_traj, z_traj, color="blue", label="trajectoire
        desiree")
6  ax.plot(x_puma, y_puma, z_puma, color="red", label="mesuree")
7
8  ax.scatter(x_traj[0], y_traj[0], z_traj[0], color="blue", label="C(
        t=0)", s=50)
9  ax.scatter(x_puma[0], y_puma[0], z_puma[0], color="red", label="P(t
        =0)", s=50)
10
11 ax.set_aspect('equal')
12
13 plt.legend()
14 plt.show()

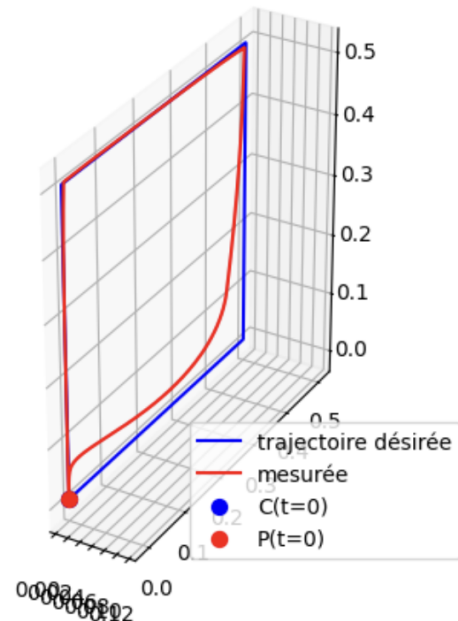
```


I Hodographes de la partie trajectoire carrée avec même condition initiale

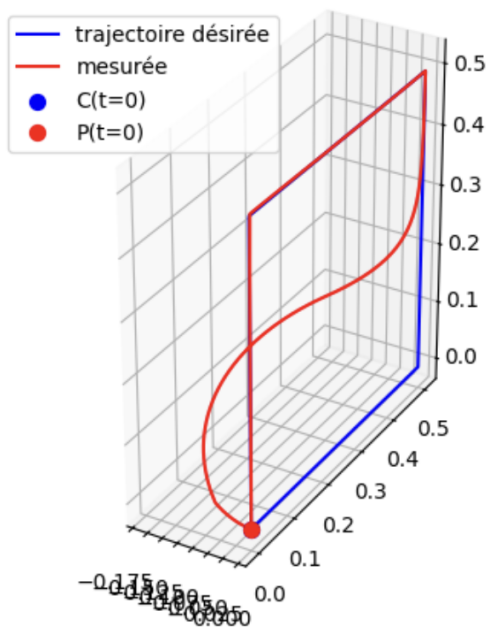
Suivi de trajectoire, $K=3$, $N=1000$



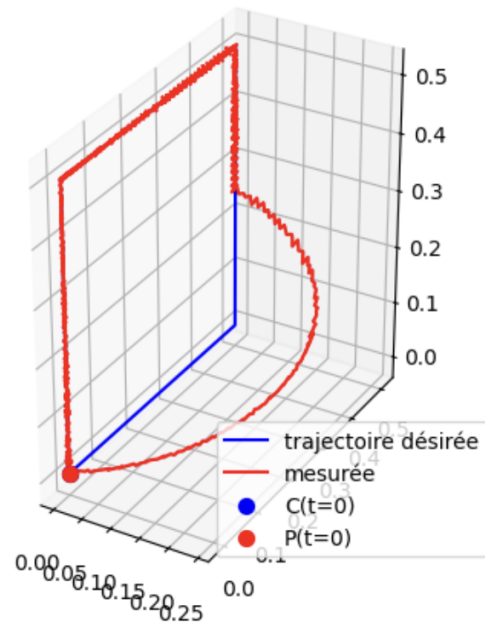
Suivi de trajectoire, $K=5$, $N=1000$



Suivi de trajectoire, $K=10$, $N=1000$

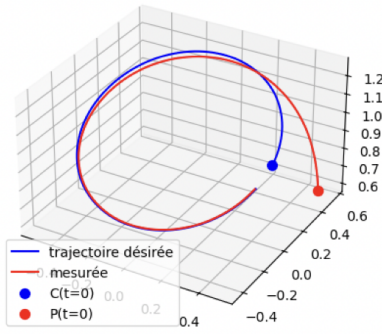


Suivi de trajectoire, $K=1000$, $N=1000$

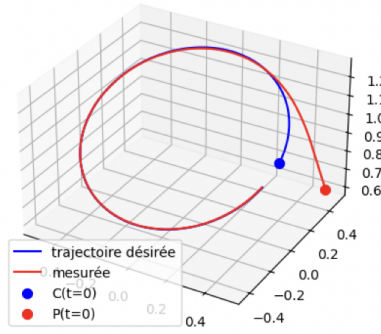


J Hodographes de la partie cercle conditions initiales différentes

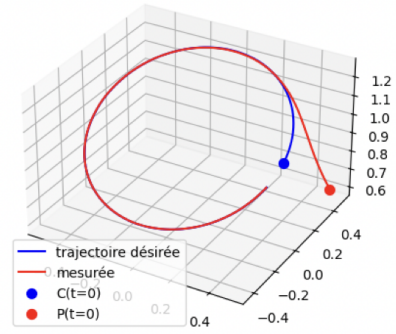
Suivi de trajectoire, $K=1$, $N=1000$



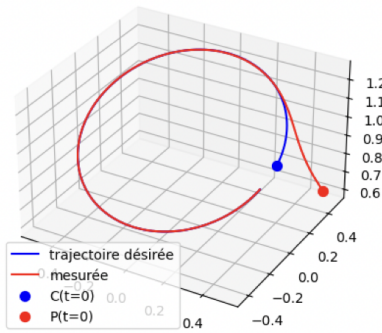
Suivi de trajectoire, $K=2$, $N=1000$



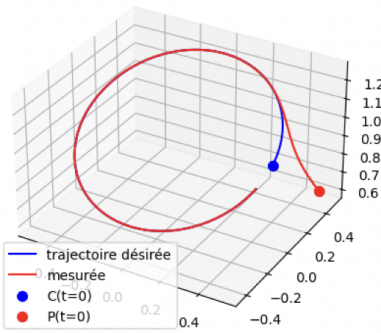
Suivi de trajectoire, $K=3$, $N=1000$



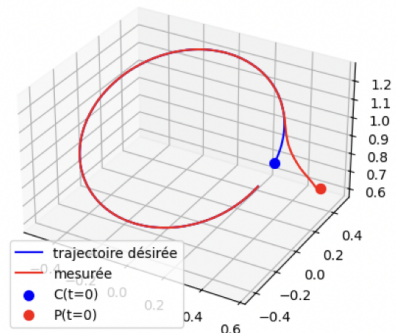
Suivi de trajectoire, $K=4$, $N=1000$



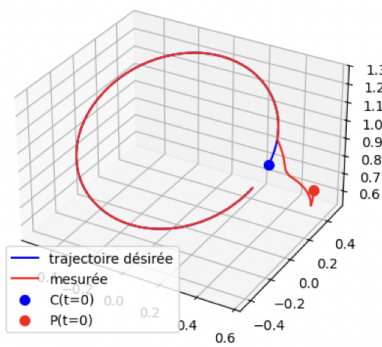
Suivi de trajectoire, $K=5$, $N=1000$



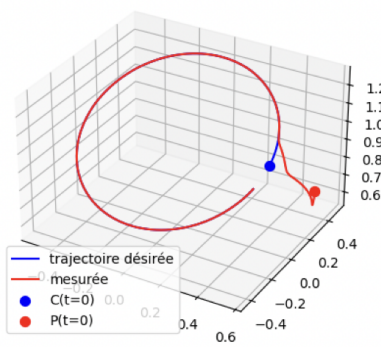
Suivi de trajectoire, $K=10$, $N=1000$



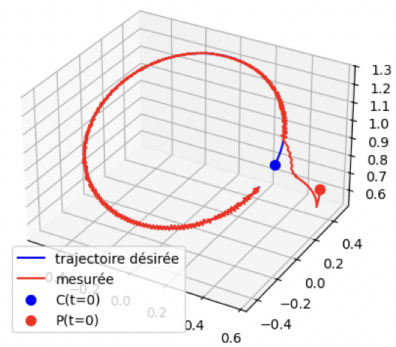
Suivi de trajectoire, $K=100$, $N=1000$



Suivi de trajectoire, $K=50$, $N=1000$



Suivi de trajectoire, $K=1000$, $N=1000$



K Erreur partie cercle conditions initiales différentes pour $K=100$

